2110211 Introduction to Data Structure

Sorrawee Worawichayawiwat

May – June, 2026

Lecture 1

C++ Intro

C++ Workflow

Source Code (`*.cpp`) ⟶ Compiler (`gcc`) ผ่าน Code::Blocks ⟶ Executable File (`*.exe`)

Hello World

```cpp
#include <iostream>

int main() {
  std::cout << "Hello, CP!" << std::endl;
  return 0;
}
```

C++ from Java & Python

ตัวแปร

- ต้องประกาศ
- ต้องระบุประเภท
- ตัวแปร โดยปรกติเป็น “กล่อง” ที่เก็บข้อมูล ไม่ใช่ของที่ “อ้างถึง” ข้อมูล

```cpp
#include <iostream>

using namespace std;

int main() {
  int x;
  x = 10;
  bool y = true;
  cout << y << endl;
  cout << (x+20) << endl;
  // x = y; // ไม่ได้
}
```

C++, if, for, while, function

If

- ต้องมีวงเล็บ ตรงเงื่อนไข
- ใช้ `{}` เป็นตัวระบุ suite (ใน c++ เรียก block)
- ไม่มี `elif` ต้องใช้ `else if`

```cpp
int main() {
  int age;
  cout << "Please enter your age:";
  cin >> age;
  if ( age < 5 ) {
    cout << "You are a kid!\n";
  } else if ( age < 100 ) {
    cout << "You are not old!\n";
  } else {
    cout << "You live long!\n";
  }
  return 0;
}
```

For loop

- เหมือน Java
- ประกอบด้วย 3 ส่วน
  - initial
  - condition
  - iteration

```
#include <iostream>

using namespace std;

int main() {
  for (int i=0; i < 10; i++) {  // i = i + 1
    cout << "i = " << i << endl;
  }
  return 0;
}
```

While loop

```
int main() {
  int x = 0;
  while ( x < 10 ) {
    cout << "x = " << x << endl;
    x++;
  }
  return 0;
}

int main() {
  int x = 20;
  do {
    x--;
    cout << x << endl;
  } while ( x > 10 )
}
```

Function

- Pass by value vs pass by reference

```
void pass_by_value(int x) {
  cout << "X is " << x << endl;
  x = 30;
}

void pass_by_reference(int &x) {
  cout << "X is " << x << endl;
  x = 40;
}

int main() {
  cout << "Pass by Value, direct" << endl;
  pass_by_value(10);  // X is 10
  cout << endl;

  int x = 20;
  cout << "Pass by value, variable" << endl;
```

```
  pass_by_value(x);  // X is 20
  cout << "outside PbV function x = " << x << endl;  // 20
  cout << endl;

  cout << "Pass by reference" << endl;
  pass_by_reference(x);  // X is 20
  cout << "outside PbR function x = " << x << endl; // 40

  // the following line cannot be compiled
  // because we need reference
  // pass_by_reference(20);
}
```

Lecture 2

Array & Vector

Array

- Two types: Static and Dynamic
- Raw Data
- Fast random access
- Static = size cannot be changes
- Dynamic = can change size

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<int> a = {2,3};
  vector<bool> b = {true,false,true};

  a.push_back(10);
  a.push_back(20);

  a.insert( a.begin(), 99);
  a.insert( a.end(), -5);

  sort(a.begin(),a.end());

  cout << "a is ";
  for (size_t i = 0; i < a.size(); i++) {
    cout << a[i] << " ";
  }
  cout << endl;

  cout << "b is ";
  for (size_t i = 0; i < b.size(); i++) {
    cout << b[i] << " ";
  }
  cout << endl;
}
```

Word Count Problem

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <set>
#include <algorithm>
#include "Tokenizer.h"

using namespace std;

void printUniqueWords1(string filename);
void printUniqueWords2(string filename);
void printUniqueWords3(string filename);
void printUniqueWords4(string filename);
void printWord(string filename);

int main() {
  string filename = "test.txt";
  printWord(filename);
  printUniqueWords1(filename);
  printUniqueWords2(filename);
  printUniqueWords3(filename);
  printUniqueWords4(filename);
  return 0;
}

bool search(string words[], int n, string w) {
  for (int i = 0; i < n; i++) {
    if (words[i] == w) return true;
    n++;
  }
}

void printWord(string filename) {
  int n = 0;

  Tokenizer tokenizer(filename);
  while(tokenizer.hasNext()) {
    string token = tokenizer.next();
    n++;
  }
  tokenizer.close();
  cout << "A total of " << n << " words" << endl;
}

void printUniqueWords1(string filename) { // wastes too much memory
  string words[100000];
  int n = 0;

  Tokenizer tokenizer(filename);
  while(tokenizer.hasNext()) {
    string token = tokenizer.next();
    if (!search(words,n,token)) words[n++] = token;
  }
```

```
  tokenizer.close();
  cout << "A total of " << n << " words" << endl;
}
```

Dynamic Array

```
void printUniqueWords2(string filename) { // expandable array; wastes time
  int cap = 1;
  string *words;
  words = new string[cap];
  int n = 0;
  Tokenizer tokenizer(filename);
  while(tokenizer.hasNext()) {
    string token = tokenizer.next();
    if (!search(words,n,token)) {
      if (n == cap) {
        string *a = new string[2*cap];
        for (int i=0; i<n; i++) a[i] = words[i];
        delete[] words;
        words = a;
        cap *= 2;
      }
      words[n++] = token;
    }
  }
  tokenizer.close();
  cout << "A total of " << n << " words" << endl;
}
```

Word Count by Vector & Set

Vector

```
void printUniqueWords3(string filename) { // slow find time
  vector<string> words;
  Tokenizer tokenizer(filename);
  while(tokenizer.hasNext()) {
    string token = tokenizer.next();
    if (words.end() = find(words.begin(), words.end(), token))
      words.push_back(token);
  }
  tokenizer.close();
  cout << "A total of " << n << " words" << endl;
}
```

Set

```
void printUniqueWords4(string filename) { // quick find existence
  set<string> words;
  Tokenizer tokenizer(filename);
  while(tokenizer.hasNext()) {
    string token = tokenizer.next();
    // if (words.end() == find(words.begin(), words.end(), token)) // slow find
    if (words.end() == word.find(token)) // if can't find then return word.end()
      words.insert(token);
  }
  tokenizer.close();
  cout << "A total of " << n << " words" << endl;
}
```

Lecture 3

std::pair

Basic

```cpp
#include <iostream>

int main() {
  std::pair<int,float> x;
  std::pair<int,float> y;

  x.first = 10;
  x.second = 20.65;

  std::cout << x.first << " "; // 10
  std::cout << x.second << std::endl; // 20.65

  y = x;

  std::cout << y.first << std::endl; // 10
  std::cout << y.second + 20 << std::cout; // 40.65
}
```

Pair reserves 2 spaces in RAM for `first` and `second` elements. When you use `y = x`, it copies values of `x` to `y`. You need to build template by specifying data types in a pair.

Initialize

```cpp
#include <iostream>

int main() {
  // default constructor
  std::pair<std::string,bool> p; // default of that type: {"", false}
  std::cout << "default [" << p.first "] [" << p.second << "]" << std::endl;

  // initialize by { }
  std::pair<std::string,bool> p1 = { "somchai", true };

  // create pair without specifying type by "make_pair"
  std::pair<bool,int> p2;
  p2 = std::make_pair(false,10);

  std::pair<bool,int> p3(p2); // must be same type p3 = p2

  // more complex pair
  std::pair< std::pair<float,int>, std::string >  p4 = { {20.5, -3}, "abc"};
  std::cout << p4.first.first << " " << p4.first.second << " " << p4.second <<
std::endl; // 20.5 -3 abc
}
```

Array inside Pair

```cpp
#include <iostream>

int main() {
    std::pair<std::string*,std::pair<int, float>> p;

    p.first = new std::string[5];
```

```
    p.first[0] = "a";
    p.first[1] = "b";
    p.first[2] = "c";

    p.second = std::make_pair(10,1.5);

    std::cout << p.first[0] << " " << p.second.first << " " << p.second.second;
}
```

Template

- Template allows "same code, different data types"
- `std::pair` is a "class template"
  - In generic term, pair is defined as `pair<T1, T2>`
- To use `std::pair`, we must supply "Type information" to the template what `T1` and `T2` should be.

STD and Namespace

- The "Fullname" of `cout` is `std::cout`
- `std` is a namespace
- We need to use fullname
  - Too lazy? use "`using`" keywords

```
#include <iostream>

// this tells C++ that when we say cout, we mean std::cout;
using std::cout;

int main() {
  // we still need to use std::endl
  cout << "Yes" << std::endl;
}
```

```
#include <iostream>

// this tells C++ that when we say something that is does not understand, C++ should
try to use std as its namespace

// this is VERY BAD PRACTICE in real world.
// 10/10 not recommend
// ... but it's ok for this class
using namespace std;

int main() {
  cout << "Yes" << endl;
}
```

std::vector

A linear storage of a single data type

Basic

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
```

```
  vector<int> v1;
  cout << "Size of v1 is " << v1.size() << endl; // 0

  vector<int> v2 = {2,3,4};

  cout << v2[1] << endl; // 3
  v1 = v2;
  v1[0] = 20;

  cout << v1[0] << ", " << v1[1] << v1[2] << endl; // 20, 3, 4

  v1.push_back(99);
  cout << v1.size() << endl;
}
```

- Vector starts with empty element
- Use `size()` to get the number of element
- Use `empty()` to check if a vector has any element

Initialize

- With specific size
- With specific size and starting value

```
#include <iostream>
#include <vector>

using namespace std;

void print_vector(vector<float> v) {
  for (size_t i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
  }
  cout << endl;
}

int main() {
  vector<float> v1(10); // size = 10
  print_vector(v1);

  vector<float> v2(5,3.55); // size = 5, default = 3.55
  print_vector(v2);

  vector<float> v3(v2);
  print_vector(v3);
}
```

Vector Access & Overflow Problem

Access

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
  vector<float> v1(2);
  vector<float> v2(2);
```

```
  cout << "-- v1 --" << endl;
  for (int i = 0; i < 7; i++) {
    v1[i] = i; // this automatically assigns value
    cout << i << ": " << v1[i] << endl; // print nonsense if out of range
  }
  cout << "-- v2 --" << endl;
  for (int i = 0; i < 7; i++) {
    cout << i << ": " << v2[i] << endl; // print "4 5 6" due to buffer overflow (take
space of v1)
  }
  cout << "using at" << endl;
  v2.at(1) = 99;
  // this will cause exception 'std::out_of_range'
  for (int i = 0; i < 7; i++) {
    cout << ": " << v2.at(i) << endl;
  }
}
```

- Operator [] won't check for 'out-of-range'
  - Reading, writing beyond size is undefined
  - Might crash
  - Grader will give 'x'
- at () will check bound
  - But slower

You should be careful of using spaces, it can access illegal memories and cause weird behaviors.

Modifying Vector

Resizing

- Resize change the size
  - Enlarge will fill with default

```
#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> v) {
  cout << "Size of V is " << v.size() << ":";
  for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
  cout << endl;
}

int main() {
  vector<int> v(3,10);
  print(v); // 10,10,10,
  v.resize(6);
  print(v); // 10,10,10,0,0,0,
  v.resize(1);
  print(v); // 10,
  v.resize(7);
  print(v); // 10,0,0,0,0,0,0
}
```

Modify

- `pop_back`
  - Erase last element
- `insert(position,value)`
- `erase(position)`
- Careful!
  - Both insert and erase position must be valid
  - If not vald, it can crash

```
#include <iostream>
#include <vector>

using namespace std;

void print(vector<int> v) {
  cout << "Size of V is " << v.size() << ": ";
  for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
  cout << endl;
}

int main() {
  vector<int> v(3,8); // v.begin() is called iterator
  v.insert( v.begin(), 1);   // 1, 8, 8, 8
  v.insert( v.begin()+3, 2); // 1, 8, 8, 2, 8
  v.insert( v.end(), 3);     // 1, 8, 8, 2, 8, 3
  print(v);
  v.erase(v.begin();         // 8, 8, 2, 8, 3
  v.erase(v.begin()+2);      // 8, 8, 8, 3
  print(v);
  v.pop_back();              // 8, 8, 8
  print(v);
}
```

std::find & std::sort

Find

- `find(a,b,c)`
  - a and b are position (iterator)
  - find c from a to *before* b
  - if not found `return b`
- Must `#include <algorithm>`

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<int> v = {9,-1,3,7,5,2,1,4};

  int x = 5;
  if ( find(v.begin(), v.end(), x) != v.end() ) {
    cout << "found" << endl;
  } else {
    cout << "not found" << endl;
```

```
  }

  if (find(v.begin(), v.begin()+3, 3) != v.begin()+3) cout << "FOUND" << endl;

  // How many "YES" will be printed? (CHEAT QUESTION)
  if (find(v.begin()  , v.begin()+2, x) != v.end()) cout << "YES" << endl; // YES
  if (find(v.begin()  , v.begin()+4, x) != v.end()) cout << "YES" << endl; // YES
  if (find(v.begin()+4, v.begin()+2, x) != v.end()) cout << "YES" << endl; // YES
  if (find(v.begin()+4, v.begin()+8, x) != v.end()) cout << "YES" << endl; // YES
  // First 3 can't even give v.end(), last did find so it gave that position instead
}
```

Sort

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

void print(vector<int> v) {
  cout << "Size of V is " << v.size() << ": ";
  for (int i = 0; i < v.size(); i++) cout << v[i] << ", ";
  cout << endl;
}

int main() {
  vector<int> v = {9,-1,3,7,5,2,1,4};

  print(v);  // 9, -1, 3, 7, 5, 2, 1, 4,
  sort(v.begin()+2, v.begin()+6);
  print(v);  // 9, -1, 2, 3, 5, 7, 1, 4,
}
```

Functions that Work with Vector

- `sort`
- `find`
- `min_element`
- `max_element`
- `lower_bound`
- `upper_bound`

Need to use *iterator*

Lecture 4

Vector Iterator

Iterator

- Iterator is a *pointer to position*
- First element is `begin()`
- The one *after* the last element is `end()`
- For `insert`, valid position is from `begin()` to `end()`, inclusive
- For `erase`, valid position is from `begin()` up to *before* `end()`
- Different vector, different iterator

- `v1.end()` is not the same as `v2.end()`

Iterator Arithmetic

- We can add by integer to an iterator
- We can subtract two iterators of the same type
- We can use increment (++) and decrement (--)

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  vector<int>   v1 = {0, 10, 20, 30, 40, 50, 60, 70, 80};
  vector<float> v2 = {0.2, -4, 0.13, 3.14, 2.73};

  vector<int>::iterator it1 = v1.begin() + 3; // 30
  vector<float>::iterator it2 = v2.end();     // after 2.73

  // getting value at iterator by using "*" operator
  cout << *it1 << endl;     // 30
  cout << *(it2-1) << endl; // 2.73
  cout << *it1+2 << endl;   // 32

  // iterator arithmetics
  vector<int>::iterator it3 = it1 + 2;
  cout << "data at it3: " << *it3 << endl; // 50
  cout << "difference of it3,it1: " << (it3 - it1) << endl; // 2

  vector<float>::iterator it4 = v2.end();
  it4--;
  cout << "data of it4: " << *it4 << endl; // 2.73

  // this cannot be done
  // cout << (it2 - it1) << endl;
}
```

Iterator All Elements by Iterator

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  vector<int>   v1 = {0, 10, 20, 30, 40, 50, 60, 70, 80};
  vector<float> v2 = {0.2, -4, 0.13, 3.14, 2.73};

  cout << "----v1----" << endl;
  auto it = v1.begin();
  while (it < v1.end()) { // should use != in other iterator types
    cout << it - v1.begin() << ": " << *it << endl;
    it++;
  }

  cout << "---v2---" << endl;
  for (auto it = v2.begin(); it < v2.end(); it++) {
    cout << it - v2.begin() << ": " << *it << endl;
```

```
  }
}
```

- We can compare iterator
  - Beware!! Iterator of some data structure does not support > or < comparator
  - But for vectors, it's ok
- We can use auto keyword to automatically define a type of a variable

Iterator Invalidate

Some Functions that Use Iterators

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
  vector<int>   v1 = {3,0,1,2,4,-3,9,8};
  vector<float> v2 = {10.2, -4, 0.13, 3.14, 2.73};

  auto it1 = min_element(v1.begin(),v1.end());
  auto it2 = max_element(v2.begin()+2,v2.end());

  cout << *it1 << endl; // -3
  cout << *it2 << endl; // 3.14

}
```

- `min_element` and `max_element` get the iterator of min, max element

New Idiom that Iterate All Elements

- Shorter syntax for loop
- Can use reference operator (`&`)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

int main() {
  vector<string> v1 = {"somchai", "somying", "somsak"};

  // range-based for loop
  for (string x : v1) {
    // x is a copy of each element in v1
    cout << x << ", ";
  }

  // using reference
  // x is THE element of v1, meaning we can modify it
  for (auto &x : v1) { x.replace(0,4,"--"); }
  for (auto &x : v1) { cout << x << " ";} // --hai --ing --ak
  cout << endl;
}
```

Iterator Invalidation

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
  vector<int> v1 = {10,20};

  auto = v1.end()-1; // this points to 20
  // we resize (enlarge) v1, now [it] is invalidated
  v1.resize(10);

  // this might not crash but it actually points to somewhere not in v1
  cout << *it << endl;

  // this will crash the program
  // v1.insert(it,99);
  for (auto &x: v1) {cout << x << " ";}
}
```

- Some operation on the data structure *invalidate* existing iterators
  - For vector, this includes *addition* and *deletion* of element (including *resize*)

Set

- Storing distinct data of same type
  - The data type must be *comparable*, i.e., we can tell if a is more or less than be
- Somewhat slow insert
- Fast look up
- Fast erase
- Iterator starts from "minimum element" and goes in increasing value direction
  - Can be used to (somewhat) fast storing

Basic

- Notice that `s` does not include duplicate elements
- Also see that when we iterate, member is sorted
  - This is distinct characteristic of set

```
#include <iostream>
#include <set>
using namespace std;

int main() {
  set<int> s = {4,1,3,2,1,1,3,4};

  cout << "Size of s is " << s.size() << endl; // 4

  s.insert(10);
  s.insert(5);
  s.erase(3);

  cout << "member of s: "; // 1 2 4 5 10
  for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << " ";
  }
}
```

Somewhat Slow Insert, Iterate but Fast Find

- We will see this in the detail around last part of this course
  - For now, please believe that
    - If there are N elements in the set
    - Insert takes time directly proportional to $\log(N)$
    - Each `it++` or `it--` takes times directly proportional to $\log(N)$
    - Each find takes time directly proportional to $\log(N)$

Set Iterator

Demos Comparing Vector & Set

- Insert
  - Vector is faster that set
- Find random number
  - Set is much faster than vector

Set Iterator

- We cannot do `s.begin() + x`
  - Because, going to the next element (which is the *successor*) in set is not as fast as vector, c++ forbids `begin() + x`
    - alternatively, you can use `auto it = s.begin(); for (int i = 0; i < 0; i++) {it++;}`
  - We cannot *compare by* `>` *or* `<`
- We can still use `it++` or `it--` to go to the next or previous (*successor* or *predecessor*) or x

```
#include <iostream>
#include <set>
using namespace std;

int main() {
  set< pair<string,int> > s = { {"somchai",5}, {"abc",6}, {"abcd",-3}, {"somchai",-4},
    {"z",0}, {"z",-1}, {"z",9} };

  for (auto &x : s) {
    cout << x.first << "," << x.second << endl;
  }

  cout << "-- find --" << endl;
  auto it = s.find({"z",-1});
  cout << (*it).first << "," << (*it).second << endl;
  it--;
  it--;
  cout << it->first << "," << it->second << endl;
  it++;
  cout << it->first << "," << it->second << endl;
}
```

Additional Function

- `set.lower_bound`
- `set.upper_bound`
- `set.count`

std::map

Associative data structure with same property as set

Map

- Is very similar to Python's `dict` in usage
- Is internally implemented as a set with "pair" data type
  - *Same properties, same limitations* as set but more convenience to use as associative data structure
- Associative (mapping) between a *Key Type* and a *Mapped Type*

Basic

```cpp
#include <iostream>
#include <map>
using namespace std;

int main() {
  // map between "Key Type" string and "Mapped Type" int
  map<string,int> m;
  m["somchai"] = 10;
  m["somying"] = -5;
  cout << "Size = " << m.size() << endl; // 2

  // accessing unseen key create a map with default value
  cout << "m[\"xxx\"] = " << m["xxx"] << endl; // 0
  // each element is a pair of key type and mapped type
  for (auto it = m.begin(); it != m.end(); it++) {
    cout << it->first << " is mapped to " << it->second << endl;
  } // somchai: 10, somying: -5

  // this will create mapping "abc" to 0 first and then increase it
  m["abc"]++;
  cout << "now size = " << m.size() << endl; // 4
  for (auto &x : m) {
    cout << x.first << " is mapped to " << x.second << endl;
  } // abc: 1, somchai: 10, somying: -5, xxx: 0
}
```

Checking If Map Has This Key?

- Use `find()`

```cpp
#include <iostream>
#include <map>
using namespace std;

int main() {
  map<int,string> m;
  m[1] = "somchai";
  m[99] = "nattee";

  int k = 99;
  map<int,string>::iterator it;
  if ((it = m.find()) != m.end()) {
    cout << "Key " << it->first << " is mapped to " << it->second << endl;
  } else {
    cout << "Key " << k << " does not exist in m." << endl;
  }

  // this is not the correct way to check if key exists why??
  if (m[k] != "") {
    cout << "exists" << endl;
```

```
  } else {
    cout << "does not exists" << endl;
  }
}
```

Requirement of std::set and std::map

- Set data type and map key type must be comparable
  - We must be able to compare *order* of two elements
- Type that we can use directly
  - int, bool, float, string, double, char, ... and most of other numerical data type
  - Pair can also be used if both the types of first and second are comparable
    - Pair compare first then second

Practice Reading C++ Docs

- Both map and set has *insert* and *erase* function
- What is the return value of both function of each data structure?
  - For `set<int> s`, can we do `s.erase(20)`
  - For `map<string,bool> m`, can we do `m.erase("Somchai")`??
- If we wish to erase element from index 3 to index 4096 in a vector
  - Is there any function from vector that we can easily use?

Pair-Sum Problem

- Pair Sum
  - Given an array of integers, out task is to find whether there exists a pair of elements in the array such that their summation equal to X
- Input:
  - Array of integers (our main array); this is a large array
  - M values of X, for each value X, we have to determine if a pair whose sum equal to X exists.
- Output:
  - For each value of X, print "YES" if we found such pair; print "NO" otherwise

Answer

```
#include <iostream>
#include <array>
#include <algorithm>
#include <cmath>

using namespace std;

bool check(int a[], int n, int X);

int main() {
  int a[15] = {323, 232, 138, 230, 437, 176, 127, 111, 329, 1, 296, 12, 3, 2, 6};
  int n = sizeof(a) / sizeof(a[0]);
  int X = 10000;

  cout << check(a,n,X);
}

bool check(int a[], int n, int X) {
  for (int i=0; i<(pow(2,n)); i++) {
    int t = X;
    for (int j=0; j<n; j++) {
      if (i / ((int) pow(2,j+1)) % 2 == 0) {
        t -= a[j];
```

```
      }

      if (0 == t) {
        cout << "YES" << endl;
        return true;
      }
      if (0 > t) {break;}
    }
  }
  cout << "NO" << endl;
  return false;
} // My stupid solution where I brute force every possible combinations
```

Lecture 5

Stack

การเพิ่ม / ลบข้อมูลในกองซ้อน

- ข้อมูล เข้าหลัง ออกก่อน (Last_In First-Out)

กองซ้อน : stack

```
bool            empty();
unsigned        size();
T               top();            // ดูบนสุด
void            push(T element); // ใส่ข้อมูล
void            pop();            // เอาออก
```

ข้อควรระวัง

- ใช้ `#include <stack>`
- `top()` ตอนไม่มีข้อมูล → พัง
- `pop()` ตอนไม่มีข้อมูล → พัง
- stack ไม่มี iterator
  - ดูได้เฉพาะข้อมูลที่เพิ่งใส่เข้าไปล่าสุด (`top`)
  - ไม่มี `begin()`, `end()` ให้ใช้
  - ถ้าอยากมาดูข้อมูลทุกตัวใน stack ต้องจำใจ pop ข้อมูลออกมา

ตัวอย่างการใช้งาน Stack

- การตรวจสอบโครงสร้างแบบซ้อนกัน เช่น การใส่วงเล็บ `(){}[]`...
- การจัดเก็บตัวแปรและการทำ function calls
- การประมวลผลนิพจน์ทางคณิตศาสตร์ postFix → Discrete Math
- การทำ undo/redo
- การค้นคำตอบแบบ depth-first search
- ...

Parentheses Checking

การตรวจสอบการใส่วงเล็บ

- ถูก : `( { ( ) [ { } ] } )`
- ผิด : เปิดปิดไม่ตรงกัน `( { ] )`
- ผิด : มีปิดมากเกินไป `( { ( ) } ) ) }`
- ผิด : มีเปิดมากเกินไป `( { ( ) }`
- วิธีทำ
  - อ่านมาทีละตัว

- ถ้าเป็นวงเล็บเปิด ให้ push ลง stack
- ถ้าเป็นวงเล็บปิด ให้ pop จาก stack มาตรวจสอบว่าเป็นวงเล็บเปิดที่ตรงกันกับวงเล็บปิดหรือไม่
- เมื่อใดที่อยาก pop ถ้า `isEmpty` แสดงว่า ปิดมีมากไป
- เมื่ออ่านเสร็จหมด stack ยังมีข้อมูล แสดงว่า เปิดมีมากไป

Function Call Stack

Using Stack in Java Virtual Machine

- jvm uses java stack to store
  - state of method calls
  - parameters and local variables
  - temporary memory for computation (operand stack)

```
public class StackFrame {
  public static void main(String[] args) {
    a(3,2);
    b(5);
  }
  static void a(int x, int y) {
    int z = x/y;
    b(z);
  }
  static void b(int x) {
    ++x;
  }
}
```

We make a stack frame of function calls, each element consists of arguments and returning argument of the function. Function `main` contains `args` and RA `JVM` and calls `a`, `a` will be added to stack and contains `z, y, x`, and RA `04:`, then it calls `b`. After `b` is finished, it will come back to line `9` and popped itself. It will do `a` and go to line 4, and so on. Arguments in each functions are separated variables.

```
#include <iostream>

using namespace std;

void test(int x) {
  if (x > 0) {
    int y = x-1;
    cout << "I am test(" << x << "). continue... " << endl;
    test(y);
    cout << "now x is " << x << endl;
  } else {
    cout << "I am test(0). stop"
  }
}

int main() {
  test(4);
  return 0;
}
```

Output

```
I am test(4). continue...
I am test(3). continue...
I am test(2). continue...
I am test(1). continue...
```

```
I am test(0). stop
now x is 1
now x is 2
now x is 3
now x is 4
```

Postfix Evaluation

Infix and Postfix Expressions

- infix (เติมกลาง)
  - `a + b * c / d - 2`, `(a + b) * c / (d - 2)`
  - use order of operations
  - use parentheses
- postfix (เติมหลัง)
  - `a b c * d / + 2 -`, `a b + c * d 2 - /`
  - order is from left to right
  - parentheses may be omitted

```
1   2   +   3   *   4   1   -   /
        3   3   *   4   1   -   /
                9   4   1   -   /
                9           3   /
                                3
```

Calculate Postfix Expressions with Stack

- Value of `2 3 + 4 5 - 6 * +` ?
- Use stack to help
- Solution
  - Check each element in postfix from left to right
  - If it is operand, `push` stack
  - If it is operator, `pop` elements from stack as many as operator needs, then `push` the result
  - After complete, answer will be at the top of stack

Convert Infix to Postfix

Use Stack to Convert

- input : infix expression
- output : postfix expression
- Steps
  - Read each element in infix
  - If it is operand, move it to the back of output
  - If it is operator
    - May `pop` operator to the end of output
    - `push` operator to stack
  - If all infix is read
    - `pop` all operators to the end of input

```
string infix2postfix(string &infix) {
  int n = infix.length();
  string postfix = "";
  stack<char> s;
  for (int i=0; i<n; i++) {  // Read each element
    char token = infix[i];
    if (token เป็น operand) {
      postfix += token;  // if operand, add to output
    } else {
```

```
      while ( ???? ) {
        postfix += s.top();  // if operator, pop stack to result,
        s.pop();
      }
      s.push(token);         // then push new operator
    }
  }
  while (!s.empty()) {postfix += s.top(); s,pop();}
  return postfix;
}
```

Priority of Operators

```
input   2   *   3   +   4

read     output     stack
 2
 *          2
 3          2         *
 +         2 3        *   (* comes before and is more important than +)
 4        2 3 *       +
         2 3 * 4      +
        2 3 * 4 +
```

```
input   2   +   3   *   4

read     output     stack
 2
 +          2
 3          2         +
 *         2 3        +
           2 3       * +
          2 3 *       +
         2 3 * +
```

Code to Compare Priority

```
string infix2postfix(string &infix) {
  int n = infix.length();
  string postfix = "";
  stack<char> s;
  for (int i=0; i<n; i++) {
    char token = infix[i];
    if (priority.find(token) != priority.end()) {
      postfix += token;
    } else {
      int p = priority[token];  // Get priority of the new operator
      while ( !s.empty() && priority[s.top()] >= p ) {  // lazy evaluation; left first
        postfix += s.top();  // pop if top operator is not less important than new op
        s.pop();
      }
      s.push(token);
    }
  }
  while (!s.empty()) {postfix += s.top(); s,pop();}
  return postfix;
}
```

Lazy Evaluation

Setting Priority of Operators

- ^ represents power 7
- ^ before + - * /
- * / before + -
- * same as / 5
- + same as - 3

```
map<char,int> priority =
    {{'+',3},{'-',3},{'*',5},{'/',5},{'^',7}};
```

Complications with Parentheses

- In parentheses is like sub-expression
- Open parentheses always push (very high priority)
- Close parentheses is very low priority
- If found, pop until sees open parentheses

```
input   2   *   (   3   +   4   )

read     output     stack
 2
 *          2
 (          2          *
 3          2         ( *
 +        2 3         ( *
 4        2 3       + ( *
 )      2 3 4       + ( *  (pop all)
      2 3 4 + *
```

```
int p = priority[token];
while ( !s.empty() && priority[s.top()] >= p ) {
  postfix += s.top();
  s.pop();
}
s.push(token);

int p = outpriority[token];
while ( !s.empty() && inpriority[s.top()] >= p ) {
  postfix += s.top();
  s.pop();
}
if (token == ')') s.pop(); else s.push(token);
```

Priority of Operators with Parentheses

	+	-	*	/	^	(	)
In expression	3	3	5	5	7	9	1
In stack	3	3	5	5	7	0	

Right to Left Operator

	+	-	*	/	^	(	)
In expression	2	2	4	4	8	9	1
In stack	3	3	5	5	7	0	

Note Power in expression is more important than in stack

```
input   2   +   3   -   4   ^   5   ^   6

read     output     stack
 2
 +          2
 3          2         +
 -        2 3         +
 4        2 3 +       -
 ^       2 3 + 4     ^ -
 5       2 3 + 4     ^ -
 ^      2 3 + 4 5    ^ -
 6      2 3 + 4 5   ^ ^ -
      2 3 + 4 5 6  ^ ^ -
   2 3 + 4 5 6 ^ ^ -
```

Conclusion

- Implementations of stack to solve problems
- Main operations: `push` / `pop` / `top`
- Make stack with array
- If reserve enough space, all runs is $\Theta(1)$; constant time

Lecture 6

Queue

- Just like a normal queue in real life.
  - First-in-First-Out data structure
  - One way in (`back` of queue), one way out (`front` of queue)
  - `push` = add data to the back of the queue
  - `pop` = remove data from the head of the queue

Basic

```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
  queue<int> q;
  q.push(1);
  q.push(2);
  q.push(3);
  while (q.empty() == false) {
    cout << q.front() << endl;
    q.pop();
  }
  cout << "-- example 2 --" << endl;
  queue<vector<int>> q2;
  vector<int> v1 = {1,2,3};
  vector<int> v2 = {99,88,-1};
  q2.push( v1 );
  q2.push( v2 );
```

```
  cout << q2.back()[1] << endl;
  cout << q2.front().size() << endl;
  auto x = q2.front();
  q2.pop();
  cout << x[0] << endl;
}
```

```
size_t      q.size()
bool        q.empty()
void        q.push(T data)
void        q.pop()
T           q.front()
T           q.back()
```

Limitation

- Same limitation as stack
  - No iterator
    - No `begin()`, `end()`
    - Can only access front and back of the queue
    - If we wish to access all members, we have to pop it all
  - Do not call `front()`, `back()`, `pop()` when the queue is empty

Radix Sort

Queue Application: Fast sorting with no comparison

Overview

- Put all data in an array
- For each digit X, from LSD to MSD
  - PUT TO QUEUE step: Sort by digit X by putting all of data from the array into B queues
    - B is the base of the number
    - For example, for a base 10 number, we will have `Queue[0]` to `Queue[9]`
    - Put into the queue labelled with that digit
  - GET FROM QUEUE step: Start form queue 0 to queue $B-1$, remove data from the queue and put back to the array

Example

Input `115 15 42 305 21 8 463`

Round 1 (digit 0), from queue

```
(0)   (1)   (2)   (3)   (4)   (5)   (6)   (7)   (8)   (9)
                                305
                                 15
       21    42   463          115                   8
```

⇒ `21 42 463 115 15 305 8`

Data is now sorted by the last digits, bacause we pop from queue 0 to 9

Round 2 (digit 1), from queue

```
(0)   (1)   (2)   (3)   (4)   (5)   (6)   (7)   (8)   (9)
  8    15
305   115    21          42          463
```

⇒ `305 8 115 15 21 42 463`

Data is now sorted by last two digits, because we goes from queue 0 to 9, which is grouped by digit 1 and the data in each queue is sorted by the last digit.

Round 3 (digit 2), from queue

```
  (0)    (1)    (2)    (3)    (4)    (5)    (6)    (7)    (8)    (9)
   42
   21
   15
    8     115           305    463
⇒  8  15  21  42  115  305  463
```

Code

```
#define base 10

int getDigit(int v, int k) {
  // return the kth digit of v (MSD is digit 0)
  int i;
  for (int i=0; i<k; i++) v /= base;
  return v % base;
}

// d = number of digits
void radixSort(vector<int> &data, int d) {
  queue<int> q[base];
  for (int k=0; k<dl k++) {
    for (auto &x : data) {
      q[getDigit(x,k)].push(x);
    for (int i=0,j=0; i<base; i++) {
      while(!q[i].empty()) {
        data[j++] = q[i].front(); q[i].pop();
      }
    }
    }
  }
}
```

Breadth First Search

Queue Application: Gotta generate 'em all

The Problem

- Given a positive integer X
- Start with a number 1, find a sequence of arithmetics operations, either "`* 3`", or "`/ 2`" that makes 1 into X
  - the `/ 2` is integer division, e.g., `5 / 3 = 1` (not 1.6666)
  - The sequence must be as short as possible
- Example
  - `10 = 1 * 3 * 3 * 3 * 3 / 2 / 2 / 2`
  - `31 = 1 * 3 * 3 * 3 * 3 * 3 / 2 / 2 / 2 / 2 / 2 * 3 * 3 / 2`

The Idea

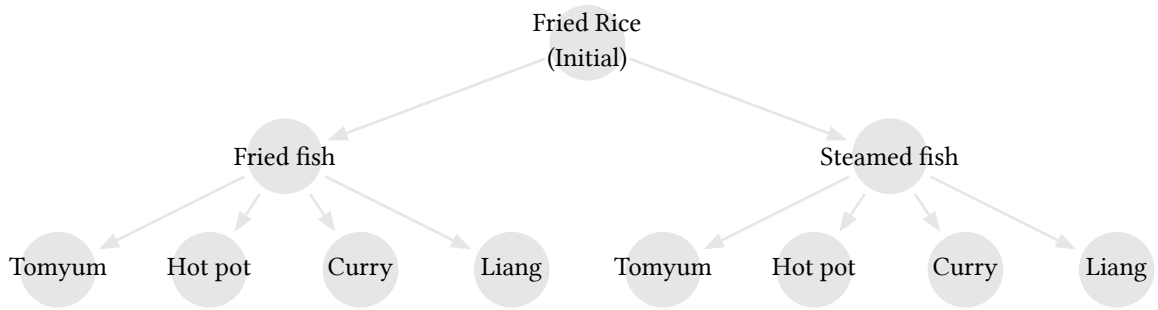
- Generate all possible sequences
  - Start with length 1, 2, 3, ... until we find one that gives X
- This is called an *exhaustsive search* algorithm
  - Systematically enumerate all possible somethings

Tree Structure (Search Tree)

- A structure to illustrate *search* algorithm
- Divide into steps

- Start with *intial solution*
- For each possible outcome (called a *state*) of each step, *generate all* proper possible *next step*
  - Also, check if the current step is what we needed
- Written as a diagram of *node* and *edge*

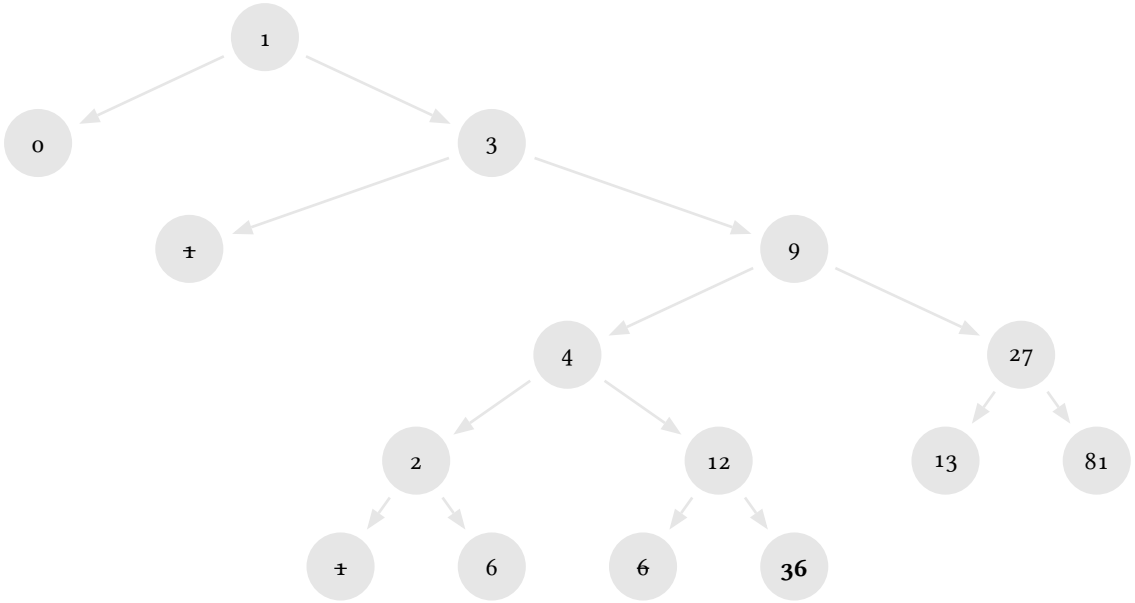
Enumerate



M3D2 Problem & Solution

Back to Our Problem

- Start with 1
- Each step is either `* 3` or `/ 2`
- Issue: might get repeated number
  - Solution: if we have found it, do not generate new step



M3D2 Code

Code

- Queue makes ordering of how we pick a state to enumerate
- From top to bottom and left to right

```
void m3d2(int target) {
  map<int, int> prev;
  queue<int> q;
  int v = 1;
  q.push(1); prev[1] = -1;
  while (!q.empty()) {
    v = q.front(); q.pop();
```

```
    if (v == target) break;
    int v2 = v/2;
    int v3 = v*3;
    if (prev[v2] == 0) {q.push(v2); prev[v2] = v;}
    if (prev[v3] == 0) {q.push(v3); prev[v3] = v;}
  }
  if (v == target) showSolution(v, prev);
}
```

Display Solution

x	0	1	2	3	4	6	9	12	13	27	36	39	40
prev[x]	1	-1	4	1	9	2	3	4	27	9	12	13	81

showSolution

```
void showSolution(int v, map<int,int>& prev) {
  string out = "";
  while (prev[v] != -1) {
    if (prev[v] * 3 == v) {
      out = "x3" + out;
    } else {
      out = "/2" + out;
    }
    v = prev[v];
  }
  out = "1" + out;
  cout << out << endl;
}
```

⇒ `36 = 1 * 3 * 3 / 2 * 3 * 3`

Lecture 7

Priority Queue

Queue with privilege

Intro

- Priority Queue is ...
  - A queue with priority
  - Item with high priority is promoted to the front of the queue
    - There is *no back of the queue*
  - Priority is defined by having more value
    - Comparison, by default, is to use `operator <`, i.e., if item `A < B` is `true`, then `B` *has higher priority*
    - We can have custom comparator
- Has the *same interface* as stack

Example

For intuitive purpose only! This is not really how priority_queue work internally

```
push(10)
  < 10          >

push(30)
  < 30 10       >
```

```
push(20)
  < 30 20 10   >

pop()
  < 20 10      >

push(15)
  < 20 15 10   >
  |
 top
```

Basic

```
size_t       q.size()
bool         q.empty
void         q.push()
void         q.pop()
T            q.top()
* there is no q.back()
```

Limitation

- Same limitation as stack, queue
  - *no iterator*
    - No `begin(), end()`
    - Use `#include <queue>`
    - Can only access `top()` of the queue
    - If we wish to access all members, we have to pop it all
  - Do not call `top(), pop()` when the queue is empty
- The data type must be *comparable* (similar to set and map)

Class

Quick Summary

- Syntax
  - class declaration must end with `;`
  - Function definition can be outside of the class
  - Access modifier is `public:, private:, protected:`
  - Constructor is a function with the same name of the class with no return type
- Object is a variable (instantiation) of a class
  - When declared, a constructor is called

Example 1

```cpp
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
  void setFullName(string name, string surname) {
    this->name = name;
    this->surname = surname;
  }
  string getFullName() {
    return "[" + name + " " + surname + "]";
  }
private:
```

```
  string name, surname;
};

int main() {
  Student a;
  Student b;
  a.setFullName("nattee", "niparnan");
  cout << a.getFullName() << endl;
  cout << b.getFullName() << endl;
}
```

Declaring class before writing functions

```
class Student {
public:
  void setFullName(string name, string surname);
  string getFullName();
private:
  string name,surname;
};

void Student::setFullName(string name, string surname) {
  this->name = name;
  this->surname = surname;
}

string Student::getFullName() {
  return "[" + name + " " + surname + "]";
}
```

Output

```
[nattee niparnan]
[ ]
```

Example 2: Constructor

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
  Student(float score) {gpax = score;}
  void setFullName(string name, string surname) {
    this->name = name;
    this->surname = surname;
  }
  string getFullName() {return "[" + name + " " + surname + "]";}
  bool is1stHonor() {return gpax >= 3.6;}
private:
  string name, surname;
  float gpax;
};

int main() {
  Student a(2.95);
  a.setFullName("nattee", "niparnan");
```

```
  cout << a.getFullName() << endl;
  if (a.is1stHonor()) {cout << "YES" endl;} else {cout << "NO" << endl;}
  // Student b; // <-- cannot compile because there is no default constructor
}
```

Operator Overloading

How C++ has a function for each operator

Overview

- Let say we write `a + b` when `a` and `b` is an object of some classes
  - This can be considered the same as calling a function `plus(a,b)`
  - C++ allows us to write a function for many operator and use it as an operator
  - For example we can write a function `times(a,b)` and let it be used as `a * b`
- This is what we called operator overloading

Instead of writing

```
Matrix multiply(Matrix a1, Matrix a2) {
  multiply(a, multiply(b,c))
}
```

We want to write

```
Matrix a, b, c
a * (b * c)
```

Example

```
#include <queue>
#include <iostream>
#include <string>

using namespace std;

// use operator + symbol that will be overloaded
string operator*(string & lhs, const int & rhs) {
  string result = "";
  for (int i=0; i<rhs; i++) {
    result = result + lhs;
  }
  return result;
}


int main() {
  string a = "abc ";
  cout << (a * 3) << endl; // this gives "abc abc abc "
}
```

- Function must be named operator followed by the operator that we will overload
- Some operator takes two parameters (such as `+, -, *, /, %`)
- Some takes one (such as `++, --, !, *, &`)

Overloading less-than operator

Using with data structure that require sorting

- We have seen several data structure that requires comparability of the data, such as set, map and priority queue
- If we want to use our class with these data structure, we need to tell them how can we use them

- There are multiple ways to achieve this
  - Let us consider operator overloading

Overloading <

- As stated earlier, set, map, and priority_queue use *operator* < to compare two elements
- It does not work if we overload *operator* >

```
class Student {
public:
  Student(float score, string a, string b) {
    name = a;
    surname = b;
    gpax = score;
  }
  bool is1stHonor() {return gpax >= 3.6;}
  // not good, now our data is public
  string name, surname;
  float gpax;
  // overloading <
  bool operator<(const Student & other) const {
    return gpax < other.gpax;
  }
};

int main() {
  Student a(2.95,"nattee","niparnan");
  Student b(4.00,"attawith","sudsang");
  cout << (a < b) << endl; // 1
  priority_queue<Student> pq;
  pq.push(a);
  pq.push(b);
  cout << pq.top().name << endl; // attawith
}
```

Custom Comparator

Why custom?

- By overloading *operator* <, we have defined default ordering of that class
- What if we need another ordering, just for this *priority_queue* only
  - For example, *Student* is ordered by *gpax* by default
  - What if we want our priority_queue to order by name instead, while keeping the *Student* default ordering elsewhere
  - Better, can we have multiple priority_queue with different ordering?
- Can be done via comparator class

Example

```
#include <iostream>
#include <string>
#include <queue>
using namespace std;

class Student {...};

// Comparator class use () overloading
class StudentByNameComparator {
public:
```

```
  bool operator()(const Student& lhs,
                  const Student& rhs) {
    return lhs.name < rhs.name;
  }
};

class GpaxThenName {
public:
  bool operator()(const Student& lhs,
                  const Student& rhs) {
    if (lhs.gpax == rhs.gpax)
      return lhs.name < rhs.name;
    return lhs.gpax < rhs.gpax;
  }
};

int main() {
  Student a(2.95,"nattee","niparnan");
  Student b(4.00,"attawith","sudsang");
  Student c(4.00,"vishnu","kotrajaras");
  cout << (a < b) << endl;  // 1
  StudentByNameComparator comp1;
  GpaxThenName comp2;

  cout << comp1(a,b) << endl; // 0; can use like function

  // Use 3 template parameters
  priority_queue<Student,          // class to compare
                 vector<Student>,  // vector of that class
                 StudentByNameComparator> pq(comp1); // comparator class
  pq.push(a);
  pq.push(b);
  cout << pq.top().name << endl;  // nattee

  priority_queue<Student,
                 vector<Student>,
                 GpaxThenName> pq2(comp2);
  pq2.push(a);
  pq2.push(b);
  pq2.push(c);
  cout << pq2.top().name << endl;  // vishnu
}
```

Another Method, lambda-function

```
#include <iostream>
#include <string>
#include <queue>
using namespace std;

int main() {
  auto compare = [](const string& lhs, const string& rhs) {
    return lhs.size() < rhs.size();
  };

  cout << "Result of compare function = " << compare("xxx","z") << endl;
```

```
  priority_queue<string,vector<string>,decltype(compare)> pq(compare);
  pq.push("somchai");
  pq.push("z");
  pq.push("abc");
  while (pq.empty() == false) {
    cout << pq.top() << endl;
    pq.pop();
  }
}
```

Output

```
Result of compare function = 0
somchai
abc
z
```

Priority Queue Template

Templating of priority_queue

- `priority_queue` requires 3 template parameters
- `priority_queue<T, Container = vector<T>, Compare = less<T>>`
- The first one is required (which is the type of the data)
- The *second* and the *third* is optional (it has default type)
  - *Second* is the container (for now, just don't think about it)
  - *Third* is the class for comparator (the class that we use to compare)
    - This one is default to `less<T>`

```
#include <iostream>
#include <string>
#include <queue>
using namespace std;

int main() {
  less<int> x;    // comparator that checks a < b
  greater<int> y; // checks a > b

  int a = 10;
  int b = 3;
  cout << x(a,b) << endl;  // 0
  cout << y(a,b) << endl;  // 1
}
```

Using Comparator for set and map

- To use custom class with set and map, we need to do the same thing, let set and map know how to sort the data
  - Either make default ordering (overload <) in the custom class
  - Or use custom comparator when declare
- For set, the declaration is `set<T, Compare = less<T>>`
- For map, the declaration is `map<Key, T, Compare = less<Key>>`

Assignment

- Is any of `vector<int>`, `set<int>`, `map<int,string>`, `queue<bool>`, `stack<vector<int>>` comparable?
  - For any class that is "YES", how it is ordered?
  - For example, if `vector<int>` is comparable, how `{1,2,3}` is compared to `{1,2,3,4}` or `{2,3,4}`

Answer

- `vector<int>` compare each element from the left; `{2,3,4} > {1,2,3,4} > {1,2,3}`
- `set<int>` compare the maximum element () of each set, if equal then look the next max
- `map<int,string>` compare by Key set, then compare max of value set
- `queue<bool>` is not comparable
- `stack<vector<int>>` is not comparable

Summary

Data Structure Summary

Data Structure	Pro	Cons	Remark
`pair<T1,T2>`	Nothings... It just a pair of two data type		
`vector<T>`	• Fast access `[ ]` • Fast append $\mathcal{O}(1)$	• Slow find $\mathcal{O}(N)$ • Slow insert, Slow Erase $\mathcal{O}(N)$	
`set<T>`	• Fast find $\mathcal{O}(\log(N))$ • Item is sorted	• Slower to just append data than vector, stack, queue $\mathcal{O}(\log(N))$ • Iterate is also slow • Takes lots of memory $\mathcal{O}(\log(N))$	Require comparator
`map<Key,T>`			• Associative data type • Also require comparator of Key_Type
`stack<T>`	• Very fast push, pop $\mathcal{O}(1)$	• Very limited functionality but has special uses	• No iterator • Order of data coming out depends on something (stack, queue depends on WHEN it is pushed, `pq` depends on value) • `pq` requires comparator
`queue<T>`			
`priority_queue<T>`	• Fast get max • Fast delete max • Data is sorted • Memory efficient	• Slower to just append data than vector, stack, queue • Very limited functionality	

More Data Structure

- C++ has more data structure not really covered right now
  - `list` is a vector with faster `input` / `erase` but does not have fast access
  - `unordered_set`, `unordered_map` are set and map that the data is not sorted but is much faster
  - `deque` is a queue that can `push`, `pop` at both ends
  - `multiset`, `multimap` are set and map that allows duplicated entries

Lecture 8

Create Your Own CP::pair

Intro

- We start writing our own data structure
  - In our own *namespace* (CP)
- We start with *pair*
  - Good introduction on writing a class
  - Some C++ feature we need to use
    - const
    - pass-by-value, pass-by-ref
    - Header file

What should we write in a class

- Class name and namespace

- Member *variables* which define what data we want to stop in the class
- Member *functions* which define behavior of the class and how member variable is manipulated
  - Include lots of special function such as *constructor*, *destructor*, *overloading*

CP:pair version 0.1 (minimal version)

- Templating
  - Parameter of a code
- No member functions
  - When we declare a pair, `T1` and `T2` must be declared as a type

```
namespace CP {
  template <typename T1, typename T2>
  class pair{
    public:
      T1 first;
      T2 second;
  };
}

int main() {
  CP::pair<int,string> p1;  // In p1, T1 is int, T2 is string
  CP::pair<bool,int> p2;    // In p2, T1 is bool, T2 is int
}
```

Capabilities

- Can hold 2 pieces of data
- Can use template
- Works with operator =
- Has copy constructor
- Cannot check equal
- Cannot check less than

```
#include <iostream>
#include <string>
using namespace std;
namespace CP {
  template <typename T1, typename T2>
  class pair{
    public:
      T1 first;
      T2 second;
  };
}

int main() {
  CP::pair<int,string> p1, p2;  // default ctor
  p1.first = 20; p1.second = "somchai";
  CP::pair<int,string> a(p1);   // copy ctor
  p2 = p1;

  cout << p2.first << "," << p2.second << endl;
}

/*
if (p1 == p2) { // won't compile
  cout << "yes" << endl;
```

```
}

if (p1 < a) { // won't compile
  cout << "yes" << endl;
}
*/
```

CP::pair version 0.2 (comparator overload)

```
#include <iostream>
#include <string>
#include <set>
using namespace std;
namespace CP {
  template <typename T1, typename T2>
  class pair{
    public:
      T1 first;
      T2 second;

      // operator
      bool operator==(const pair<T1,T2> &other) {
        return (first == other.first && second == other.second);
      }

      bool operator<(const pair<T1,T2>& other) const {  // operator< must be const
        return ((first < other.first) ||
                (first == other.first && second < other.second));
      }
  };
}

int main() {
  CP::pair<int,string> p1, p2;  // default ctor
  p1.first = 20; p1.second = "somchai";
  CP::pair<int,string> a(p1);   // copy ctor
  p2 = p1;
  cout << p2.first << "," << p2.second << endl;  // 20, somchai

  if (p1 == p2) {cout << "yes" << endl;}  // yes
  if (p1 < a) {cout << "yes" << endl;}
  set<CP::pair<int,int>> s;
  s.insert({1,2});
  // Now we can compare and use less than (and also set, map, priority queue)
  cout << s.begin()->second << endl;  // 2
}
```

Const Member

Const Parameter in Function

```
void test(int &x, const int& y) {
  x++;                // Okay; doable
  cout << y << endl;  // Okay, we only read y
  for (int i=0; i<y; i++) { // Okay, too
    cout << i << endl;
  }
```

```
  y--;  // ERROR: we promised NOT to change y
}
```

- Declared by putting a keyword const as a prefix
- Const parameter must not be modified inside the function
- Why? So that we know that the function does not modify the data
  - Especially when we pass-by-reference

Difference of Pass-by-reference && Pass-by-value

Pass-by-value

- The *arguments* can be either constants or variables
- Modifying *parameters* (variable inside the function) does not change the *argument's* value
- The *argument's* value is copied to the parameter
  - SLOWER!!! Because we have to copy

Pass-by-reference

- The *arguments* must be variables
- The *parameters* are the argument's variables
  - Modify the *parameter* also modify the *argument's* variable
- Faster

Const Member Function

```
class ccc {
public:
  int a,b;

  void inspect() const {  // This function promises NOT to change anything
    if (a < b) cout << "yes" << endl;  // Okay

    // b += 20;  // <-- NOT OKAY
  }

  void mutate() {  // This function might change something
    if (a < b) a += 10;  // Okay
  }
};

void test2(ccc& changeable, const ccc& unchangeable) {
  changeable.inspect();
  changeable.mutate();
  unchangeable.inspect();
  unchangeable.mutate();  // ERROR: attempt to change unchangeable object
}
```

- Declared by putting a keyword `const` after the function declaration
- Const member function cannot modify any member data
  - Also cannot call any other function that is not const
- Why? So that we know that the function does not modify its data

Constructor & List Initialization

Custom Constructor

- In STL spec of std::pair, it has custom constructor called *intialization constructor*

```
namespace CP {
  template <typename T1, typename T2>
  class pair{
```

```
    public:
      T1 first;
      T2 second;

      // Custom constructor, using initialization list
      pair(const T1 &a, const T2 &b) {
        first = a;
        second = b;
      }
      // When we have a constructor, a default constructor is not auto-generated

      // Operator
      bool operator==(const pair<T1,T2> &other) {...}
      bool operator<(const pair<T1,T2>& other) {...}
  };
}

int main() {
  CP::pair<int, bool> p(10,false);
  CP::pair<string, int> q("abc",42), r("",0);

  cout << (q < r) << endl;  // 0
  priority_queue<CP::pair<string,int>> pq;
  pq.push(r);
  pq.push(q);
  cout << pq.top().first << endl;  // abc

  CP::pair<string, int> x(q);
  CP::pair<string, int> y = x;

  // All below cannot be compiled
  // CP::pair<string, int> w;
  // vector<CP::pair<int, int>> v(10);
}
```

Initialization List

- Instead of writing a code to assign a value to each member, we can use *intialization list*
  - A little bit shorter code
  - Also little bit faster
  - Only way to init *const* member

```
namespace CP {
  template <typename T1, typename T2>
  class pair {
    public:
      T1 first;
      T2 second;

      // custom constructor, using initialization list
      pair(const T1 &a, const T2 &b) : first(a), second(b) { }
  };
}
```

Default Constructor

- A constructor used when we simply declare an object
- Auto-generated by init all members with its default constructor

- Won't be auto-gen if we have any other constructor

```
namespace CP {
  template <typename T1, typename T2>
  class pair {
    T1 first;
    T2 second;

    // Constructor
    pair() : first(), second() { }
    pair(const T1 &a, const T2 &b) : first(a), second(b) { }
  };
}
```

Include & Header file

Include File

- Usually, our data structure (which is written as a class) will be used by several programs in several files.
  - It is better NOT TO copy our data structure code into each each file
- Rather, put our code in a file and include it where it is needed
  - Better if we want to change it
  - Better compilation
- Introducing ".h" files

C++ header file (.h) and #include

- To put a content of one file into another file, we use `#include "filename"` keyword
- C++ will simply put the content of *filename* into where we `#include` it
- `#include` has more benefit
  - Separation of *declaration* (what it is) and *definition* (how it works)
  - Not really explored in this class
- We usually use .h for a file that we will include but that is not a rule, we can use other extension

Problem with include

a.h

```
class a {
  int m1, m2;
};
```

b.h

```
#include "a.h"

class b {
  a x;
};
```

c.h

```
#include "a.h"

class c {
  a y;
};
```

main.cpp

```
#include "b.h"
#include "c.h"
```

main.cpp (expanded)

```
class a {
  int m1, m2;
};

class b {
  a x;
};

// There are multiple copies of class a
class a {
  int m1, m2;
};

class c {
  a y;
};

int main() {
  b b1;
  c c1;
}
```

```
int main() {
  b b1;
  c c1;
}
```

Fix problem

a.h

```
#ifndef A_H
#define A_H
class a {
  int m1, m2;
};
#endif
```

b.h

```
#include "a.h"

class b {
  a x;
};
```

c.h

```
#include "a.h"

class c {
  a y;
};
```

main.cpp (expanded)

```
#ifndef A_H
#define A_H
class a {
  int m1, m2;
};
#endif

class b {
  a x;
};

// class a here is taken out of
// compilation because A_H is defined
#ifndef A_H
// #define A_H
// class a {
//   int m1, m2;
// };
#endif

class c {
  a y;
};

int main() {
  b b1;
  c c1;
}
```

Summary

Major

- Templating: allow user to write a code that works with different data type
- Constructor: a function called when we create a variable
  - Default constructor
- Operator overloading: for less-than and equality
- pass-by-ref vs pass-by-value

Minor

- Header file
- const
- List initialization

Exercise (no grader)

1. Modify `operator<` so that it compare *second* before *first*
2. Modify `operator<` so that when we call `sort(v.begin(),v.end())` where `w` is a vector of our pair, it is sorted from *Max* to *Min*
3. Write `operator!=` and `operator>=`

Final version CP.h

```
#ifndef _CP_PAIR_INCLUDED_
#define _CP_PAIR_INCLUDED_

namespace CP {
  template <typename T1, typename T2>
  class pair {
    public:
      T1 first;
      T2 second;

      pair() : first(), second() {}

      pair(const T1 &a, const T2 &b) : first(a), second(b) { }

      bool operator==(const pair<T1,T2> &other) {
        return (first == other.first && second == other.second);
      }


      // 1.
      bool operator<(const pair<T1,T2>& other) const {
        return ((second < other.second) ||
                (second == other.second && first < other.first));
      }

      // 2.
      bool operator<(const pair<T1,T2>& other) const {
        return ((first > other.first) ||
                (first == other.first && second > other.second));
      }

      // 3.
      bool operator!=(const pair<T1,T2>& other) const {
        return !(*this == other);
      }

      bool operator>=(const pair<T1,T2>& other) const {
        return !(*this < other);
      }

  };
}
#endif
```

Lecture 9

CP::vector

Our first "real" data structure

Intro

- Now we will create more complex data structure `CP::vector`
- It can store variable length array

  - Implemented as a dynamic array
- Can be accessed by `operator[]`
  - Additional operator to be overloaded
- We also have to create our own iterator
  - Implemented as a pointer

Key Idea
- Vector stored 3 things (3 member data)
  - mData: A dynamic array, large enough space to store current data and might have reserved space
  - mSize: Number of data stored
  - mCap: Size of the dynamic array (maybe large than mSize)
- If the dynamic array is full and more data is being added, we create a new dynamic array and relocate data to the new array
  - This is called expand
  - Each expansion takes very long time
- Dilemma
  - Large reserve = less often relocation but use more memory
  - Small reserve = more frequent relocation but less memory

Example
```
mSize  mCap       mData
  4     5    [10, 3, 1, 5, ]

push_back(9)

  5     5    [10, 3, 1, 5, 9]

push_back(4)

  6     10   [10, 3, 1, 5, 9, 4, , , , ]
```

How much reserve should we have?
- Whenever we need to relocate, we double the capacity that we currently have
- If we start with a vector with zero size and continuously add data one by one, for example by `push_back`
  - Each expansion will take time equal to the number of data when we relocate (this is very slow)
- By doubling the size every time we expand, we can show that on average, each addition of a data (such as `push_back`, `insert`) takes constant time!!!

Pointer
How to implement a dynamic array (and also iterator)

Pointer & Memory
- Each variable is some block in computer memory
  - Programming language just map our variable name to that block of memory
  - Programming language works with the address of that block
- Pointer variable is a variable that stor address of memory
  - Pointer needs type i.e., address of int is not the same as address of bool
  - We can use operator `&` to ask for the address of a variable
  - We can use operator `*` to ask for the data of an address

Example
```
int x, y;  // this is normal variable x and y
int *a;    // this is int pointer variable a
int *b;    // another int pointer
```

```
x = 10;     // say x is at address 3267
a = &x;     // a points to x
b = a;      // b also points to x
*b = 30;    // change the address value to 30
y = *b;     // set value of y to 30 (hard copy)

cout << &x << endl;  // 0x7ffee22885ac
cout << &y << endl;  // 0x7ffee22885a8  +4
cout << &a << endl;  // 0x7ffee22885a0  +8
cout << &b << endl;  // 0x7ffee2288598  +8
cout << sizeof(int) << endl;   // 4 bytes
cout << sizeof(*int) << endl;  // 8 bytes
```

Pointer Arithmetic

- Pointer can be added, subtracted by integer
  - It moves the address by the size of the type of the pointer
  - For example, when `X` is an int (which is 4 bytes) `X + 10` result in an address 40 bytes away from `X`
- Two pointers of the same type can be subtracted
  - The result is the address difference divided by size of the type of the pointer

```
int main() {
  bool x, y, z;
  x = 1; y = 2; z = 3;
  bool *a, *b;

  a = &x;
  b = a+2;

  cout << "&x = " << &x << endl;  // &x = 0x6afed4
  cout << "&y = " << &y << endl;  // &y = 0x6afed8
  cout << "&z = " << &z << endl;  // &z = 0x6afedc
  cout << " a = " << a  << endl;  //  a = 0x6afed4
  cout << " b = " << b  << endl;  //  b = 0x6afedc
  cout << b-a << endl;            // 2
}
```

Dynamic Array

- Dynamic Array variable of type `T` is a pointer to the starting address of consecutive block of type `T`
- Let `A` be a dynamic array of int
  - `A[x]` refer to the x^{th} block starting from `A`
- Static array works in the same way, that's why accessing `A[x]` is very fast for an array
  - It just refers to the address `A[x]` is `*(A + x * size_of(T))`

new and delete operator

- For a datatype `T`, `new T` *allocates* a block with a size of `T` and then call a *constructor* of `T`, return the address of that memory
- For a datatype `T`, `new T[n]` *allocates* `n` blocks of `T` and then call a *constructor* of `T` in each block, return the address of the first block
  - For example, we use `new int[10]` to create a dynamic int array of 10 elements
- For a pointer `X`, `delete X` calls the *destructor* and then *de-allocates* memory pointed by `X`
- For a dynamic array `X`, `delete [ ]` calls the *destructor* of all blocks in the dynamic array and *de-allocates* the memory allocated by `X`

Example

```
class test {
public:
```

```
  // constructor
  test() : data() {cout << "created" << endl;}
  // destructor
  ~test() {cout << data << " destroyed " << endl;}
  int data;
};

int main() {
  test *a, *b;   // pointers of test

  a = new test;  // created new test pointed by a
  a->data = 10;  // (*a).data = 10
  cout << a->data << endl;  // 10
  delete a;      // destroyed

  b = new test[4];  // created 4x
  b[0].data = 10;
  b[1].data = 20;
  b[2].data = 30;
  b[3].data = 40;
  delete [] b;      // deleted 4x
}
```

Memory Leak

- For everything, this is created by `new`, we must call `delete` on it
- If you do not, that memory is not deleted until all memory is used up

```
#include <iostream>
using namespace std;

void leaked() {
  int *a;
  a = new int[2000];
}
int main() {
  for (int i=0; i<1000000; i++) {
    int *a;
    cout << i << endl;
    leaked();
  }
}
// terminate called after throwing an instance of 'std::bad_alloc'
```

Smart Pointer (NOT A SUBJECT OF THIS COURSE)

- A better way is to use C++ smart pointer `shared_ptr(T)`
- Smart pointer is a pointer that can delete itself when it go out of scope
- Similar concept to Java Garbage Collection
- Still possible to have memory leak

vector.h

Version 0.1

- Start with vector that can do `push_back`, `pop_back`, and `[]`
- Also with custom constructor

```
namespace CP {
template <typename T>
  class vector {
    protected:
      T *mData;
      size_t mCap;
      size_t mSize;

      void rangeCheck(int n) {...}
      void expand(size_t capacity) {...}
      void ensureCapacity(size_t capacity) {...}
    public:
      vector() {...}
      vector(size_t capacity) {...}
      ~vector() {...}
      // access
      T& at(in index) {...}
      T& operator[](int index) {...}
      // modifier
      void push_back(const T& element) {...}
      void pop_back() {...}
  };
}
```

Basic Constructor

```
template <typename T>
class vector {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;

  public:
    vector() {
      int cap = 1;
      mData = new T[cap]();
      mCap = cap;
      mSize = 0;
    }

    vector(size_t cap) {
      mData = new T[cap]();
      mCap = cap;
      mSize = cap;
    }
```

```
vector<int> v;

mSize  mCap  mData
  0     1     [ ]
```

```
vector<int> w(5);

mSize  mCap     mData
  5     5    [0,0,0,0,0]
```

Destructor

- Since we `new mData`, we have to `delete` it
  - Or face a memory leak problem

```
    ~vector() {
      delete [] mData;
    }
};
```

Object Life Cycle

- Normal object
  - Object is created (constructor called) when declared
  - Object is destroyed (destructor called) when go out of scope
- Object created by new (both `new T` or `new T[]`) Object is created when `new` Object is destroyed when `delete`

```
class test {
public:
  // constructor
  test() : data() {cout << "created" << endl;}
  // destructor
  ~test() {cout << data << " destroyed " << endl;}
  int data;
};

int main() {
  cout << "-- Life cycle --" << endl;
  cout << "- normal cycle -" << endl;
  test u;
  u.data = 99;
  for (int i=0; i<5; i++) {
    test t;
    t.data = i*10;
  }
}
```

Output

```
-- Life cycle --
- normal cycle -
created       <-- u
created
0 destroyed
created
10 destroyed
created
20 destroyed
created
30 destroyed
created
40 destroyed
99 destroyed  <-- u
```

Accessing Data

- The return type is `T&` which is a reference
- Same deal as pass-by-refernce, this is called return-by-reference
  - So we can do `v[i] = 30` or `v[i]++`
- What is returned is actually that variable
- Also notice the difference between `at()` and `operator[]`

```
template <typename T>
class vector {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;

    void rangeCheck(int n) {
      if (n < 0 || (size_t)n >= mSize) {
        throw std::out_of_range("index out of range");
      }
    }

  public:
    T& at(int index) {
      rangeCheck(index);
      return mData[index];
    }

    T& operator[](int index) {
      return mData[index];  // return the variable
    }
};
```

Add, remove data

- push_back first check of we have reserved space
  - If not, we expand
- Then, the data is put to mData[mSize]
- Removing Data is done by just reduce the size

```
template <typename T>
class vector {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;
    void expand(size_t capacity) {
      T *arr = new T[capacity]();  // create a new dynamic array
      for (size_t i = 0; i < mySize; i++) {
        arr[i] = mData[i];  // move all data
      }
      delete [] mData;
      mData = arr;  // delete old data and point to new one
      mCap = capacity;
    }
    void ensureCapacity(size_t capacity) {
      if (capacity > mCap) {
        size_t s = (capacity > 2 * mCap) ? capacity : 2 * mCap;  // double the size
        expand(s);
      }
    }
  public:
    void push_back(const T& element) {
      ensureCapacity(mSize+1);
      mData[mSize++] = element;  // add size and put element
    }
```

```
    void pop_back() {
      mSize--;
    }
};
```

Vector Constructors

Problem of v0.1

- Copy constructor and assignment operator is incorrect
  - It is auto generate to copy all variables (but not the data it points to)
- Rule of three in c++
  - Consider destructor, copy constructor, assignment operator
  - If any of them is written in the code, we mostly need all of them
- Since c++11, it's rule of four an a half

```
int main() {
  CP::vector<int> w(5);

  for (int i=0; i<5; i++) w[i] = i*10;
  CP::vector<int> x(w);  // this creates shallow copy of w
  CP::vector<int> y = w;
  x[3] = -1;  // changes both w and y
  cout << y[3] << endl;  // -1
  cout << w[3] << endl;  // -1
}
```

v0.2, add small access functions

- `empty` and `size` also exists in other data structure
- `size_t` is non-negative integer type

```
bool empty() const {
  return mSize == 0;
}

size_t size() const {
  return mSize;
}

size_t capacity() const {
  return mCap;
}
```

v0.2 adding copy constructor & assignment operator

```
// copy constructor
vector(const vector<T>& a) {
  mData = new T[a.capacity()]();
  mCap = a.capacity();  // copies only value
  mSize = a.size();
  for (size_t i=0; i<a.size(); i++) {
    mData[i] = a[i];
  }
}

// copy assignment operator
vector<T>& operator=(vector<T> &other) {
  // protect against self-destruct
  if (mData != other.mData) {
```

```
    // delete current data
    delete [] mData;
    // copy the new data
    mData = new T[other.capacity()]();
    mCap = other.capacity();
    mSize = other.size();
    for (size_t i=0; i<a.size(); i++) {
      mData[i] = a[i];
    }
  }
}
```

Copy-and-swap

- Utilize written copy-constructor nd destructor
- Shorter code

```
// copy assignment operator using copy-and-swap idiom
vector<T>& operator=(vector<T> other) {  // notice the pass-by-value!!!
  // other is copy-constructed which will be destruct at the end of this scope
  // we swap the content of this class to the other class and let it be destructed
  using std::swap;
  swap(this->mSize, other.mSize);
  swap(this->mCap, other.mCap);
  swap(this->mData, other.mData);
  return *this;
}
```

Iterator, Insert, Erase, & Analysis

v0.3 Iterator and typedef keyword

```
template <typename T>
class vector {

  protected:
    T *mData;
    size_t mCap;
    size_t mSize;
  protected:
    typedef T* iterator;  // let iterator be type name of T pointer

    // iterator
    iterator begin() {
      return &mData[0];
    }

    iterator end() {
      return begin()+mSize;
    }
}
```

- See that pointer works just like how `std::vector::iterator` works
- In fact, iterator is actually a pointer
- `typedef` keyword allows us to map a type name
  - `CP::vector<int>::iterator` is `int*`
  - `CP::vector<bool>::iterator` is `bool*`

insert

- `push_back` actually call `insert(end(), element)`
- Question: why we need `pos`?

```
iterator insert(iterator it, const T& element) {
  size_t pos = it - begin();
  ensureCapacity(mSize+1);
  for (size_t i = mSize; i > pos; i--) {
    mData[i] = mData[i-1];
  }
  mData[pos] = element;
  mSize++;
  return begin()+pos;
}

void push_back(const T& element) {
  insert(end(), element);
}
```

erase

- See that both `insert` and `erase` also change `mSize`

```
void erase(iterator it) {
  while((it+1)!=end()) {
    *it = *(it+1);
    it++;
  }
  mSize--;
}
```

Exercise

- Read the following function and see how it works in `vector.h`
  - `resize`, `clear`
  - non-stl function
    - `insert_by_pos`
    - `erase_by_pos`
    - `erase_by_value`
    - `constains`
    - `index_of`
- Read in here

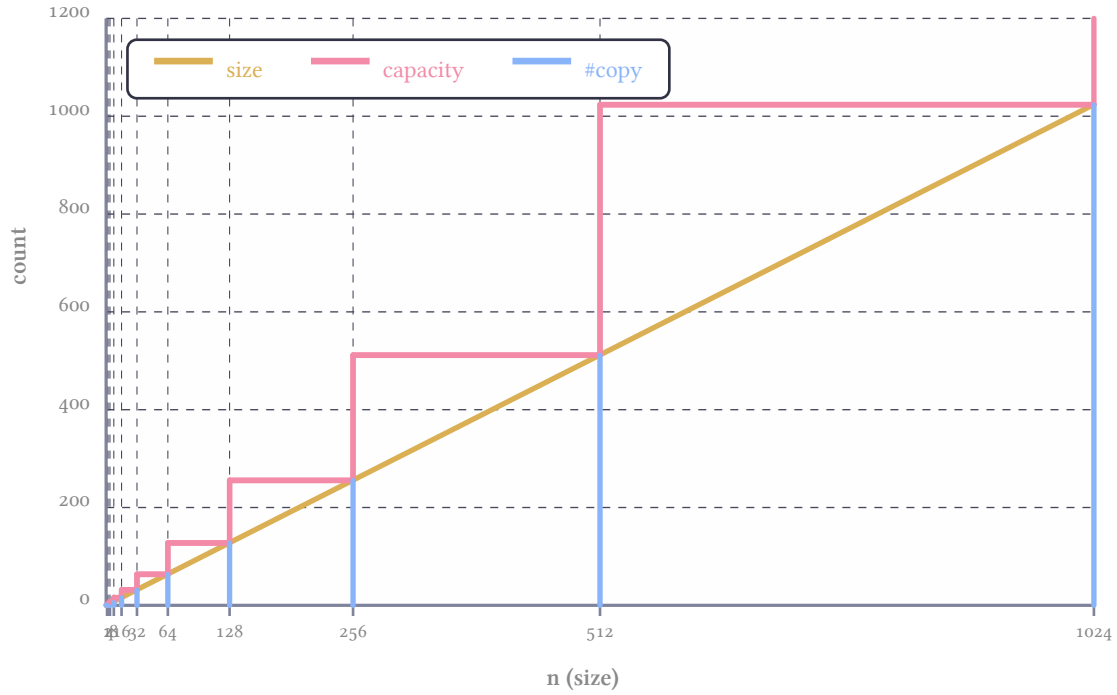
Will do this part later

Analysis of how many data is copied by push_back

- When full, `push_back` have to move all data to a new dynamic array
- `ensureCapacity` double the size

Size and Capa & Copy Count

- How much copy we need?



Lecture 10

Stack

Intro

- Now we will create less complex data structure `CP::stack`
- Just like a vector without `iterator, insert, erase, resize, at end operator[]`
  - Add `top()` which is just a shorthand of looking at the last element
- That's it, really

Key Idea

- The data is stored in the same way as a vector
  - The first element of `mData` is the bottom of stack while the last element is the top of the stack
- We just take `vector.h` and remove unnecessary function

```
|   |   stack
|   |
| A |   mSize  mCap     mData
| X |     4     5    [A, B, X, A]
| B |
| A |
|___|
```

stack.h

```
namespace CP {
  template <typename T>
  class stack {
    protected:
      T *mData;
      size_t mCap;
      size_t mSize;
      void expand(size_t capacity) {...}
      void ensureCapacity(size_t capacity) {...}
```

```
    public:
      // constructor
      stack(const stack<T>& a) {...}
      stack() {...}
      stack<T>& operator= {...}
      ~stack() {...}
      // capacity operation
      bool empty() const {...}
      size_t size() const {...}
      // access
      const T& top() const {
        return mData[mSize-1];
      }
      // modifier
      void push(const T& operator) {...}  // This is push_back
      void pop() {...}  // This is pop_back
  };
}
```

Speed of each operation

- All read operation always take constant time
  - `size()`, `top()` simply return something that is directly accessible
- All modify operation also take constant time
  - `push()` is constant on average (same as `push_back` of vector)
  - `pop()` is always constant

Stack by Vector

- Instead of writing our own function, there is another way to write a stack
- We simply use vector as our sole data member
- Benefit: code reuse
- Drawback: almost none except that we need one more layer of function call

```
namespace CP {
  template <typename T>
  class stack {
    protected:
      vector<T> v;
    public:
      // default constructor
      stack() : v() { }
      // capacity function
      bool empty() const { return v.empty(); }
      size_t size() const { return v.size(); }
      // access
      const T& top() const { return v.back(); }
      // modifier
      void push(const T& element) { v.push_back(element); }
      void pop() { v.pop_back(); }
  };
}
```

Lecture 11

CP::queue

Will the circle be unbroken?

Intro

- Queue, unlike stack, require more sophisticated technique to achieve fast performance
- We start by writing a simple class that just work (slowly)
- Then we try to improve it

Key Idea

- Just like stack, we will use the same format as vector, using dynamic array to store data
- However we have to somehow manage how we works with `front()` and `back()` of the queue

v0.1 simple implementation of the queue

- To illustrate this idea, we will use a vector as our data member
- `push(e)` is simply `v.push_back(e)`, this is fast
- `front()` is `v[0]`, `back()` is `v[v.size()-1]`, this is also fast
- `pop()` is `v.erase(v.begin())`, this is slow (always proportional to `v.size()`)
  - Unlike `std::queue` which has very fast `pop()`

```
namespace CP {
  template <typename T>
  class queue {
    protected:
      std::vector<T> v;
    public:
      // default constructor
      queue() : v() { }
      // capacity function
      bool empty() const { return v.empty(); }
      size_t size() const { return v.size(); }
      // access
      const T& front() const { return v[0]; }
      const T& back() const { return v[v.size()-1]; }
      // modifier
      void push(const T& element) { v.push_back(element); }
      void pop() { v.erase(v.begin()); }
  };
}
```

v0.1 example

```
CP::queue<int> q;     q  v
                         mSize  mCap  mData
                           0     1      ---->  [ ]
q.push(10);           q  v
                         mSize  mCap  mData
                           1     1      ---->  [10]
q.push(20);           q  v
                         mSize  mCap  mData
                           2     2      ---->  [10, 20]
q.push(30);           q  v
                         mSize  mCap  mData
                           3     4      ---->  [10, 20, 30, ]
q.pop();              q  v                          /   /
```

```
                        mSize  mCap  mData       /   /
                          2     4      ---->  [20, 30, , ]
```

v0.2 faster queue

- Add more data member `mFront`, initialized as `0`
- `push(e)` is simply `v.push_back(e)`, this is fast
- `front()` is `v[mFront]`, `back()` is `v[v.size()-1]`, this is also fast
- `pop()` is `mFront++`, this is fast
  - However, we don't really remove anything when pop

```cpp
#include <vector>

namespace CP {
  template <typename T>
  class queue {
    protected:
      std::vector<T> v;
      int mFront;
    public:
      // constructor
      queue() : v(), mFront() {}
      // capacity function
      bool empty() const { return v.empty(); }
      size_t size() const { return v.size() - mFront; }
      // access
      const T& front() const { return v[mFront]; }
      const T& back() { return v[v.size()-1]; }
      // modifier
      void push(const T& element) { v.push_back(element); }
      void pop() { mFront++; }
  };
}
```

v0.2 example

```
CP::queue<int> q;     q          v
                         mFront  mSize  mCap  mData
                           0       0     1      ---->  [ ]
q.push(10);           q          v
                         mFront  mSize  mCap  mData
                           0       1     1      ---->  [10]
q.push(20);           q          v
                         mFront  mSize  mCap  mData
                           0       2     2      ---->  [10, 20]
q.push(30);           q          v
                         mFront  mSize  mCap  mData
                           0       3     4      ---->  [10, 20, 30, ]
q.pop();              q          v
                         mFront  mSize  mCap  mData
                           1       3     4      ---->  [10, 20, 30, ]
```

Problem with v0.2

- Fast but use too many space
- Queue grows according to how many time push is called
  - regardless of how many pop is called
- The data stored in the vector can be much larger than the actual data in the queue
- Does not really work in real world

```
for (int i=0; i<1000000; i++) {
  q.push(i);
  q.pop();
}
std::cout << q.size() << std::endl;
```

Circular Queue

Final Idea

```
  mFront  mSize  mCap  mData             *
    2       2     4      ---->  [  ,   , 30, 40]
  q.push(50);
  mFront  mSize  mCap  mData             *
    2       3     4      ---->  [50,   , 30, 40]
  q.push(60);
  mFront  mSize  mCap  mData             *
    2       4     4      ---->  [50, 60, 30, 40]
  q.pop();
  mFront  mSize  mCap  mData                *
    3       3     4      ---->  [50, 60,   , 40]
  q.push(99);
  mFront  mSize  mCap  mData                *
    3       4     4      ---->  [50, 60, 99, 40]
  q.push(1);                       \   \   \
  mFront  mSize  mCap  mData     *  \   \   \
    0       5     4      ---->  [40, 50, 60, 99,  1,   ,   ,   ]
```

- We take vo.2 and reuse the area at the beginning of `mData`
  - Expand when necessary
  - Re-arrange when expand

Circular Queue

- We can think of `mData` to be circular
  - End of the last element of the `mData` is connected to the first element
- Consider i^{th} element
  - the next element is `(i+1) % mCap`
  - The previous element is `(i-1+mCap) % mCap`
  - Next `k` element is `(i+k) % mCap`

Circular Queue Implementation

queue.h

```
namespace CP {
  template <typename T>
  class queue {
    protected:
      T *mData;
      size_t mCap;
      size_t mSize;
      size_t mFront;
      void expand(size_t capacity) {...}
      void ensureCapacity(size_t capacity) {...}
    public:
      // constructor - almost the same but have to take care of mFront
      queue(const queue<T>& a) {...}
      queue() {...}
```

```
    queue<T>& operator=(queue<T> other) {...}
    ~queue() {...}
    // capacity function - same as vector
    bool empty() const {...}
    size_t size() const {...}
    // access - circular queue implementation
    const T& front() const {...}
    const T& back() const {...}
    // modifier
    void push(const T& element) {...}
    void pop() {...}
  };
}
```

Ctor, Dtor, copy

```
template <typename T>
class queue {
  protected:
    T *mData;  size_t mCap;  size_t mSize;  size_t mFront;
  public:
    // default constructor
    queue() : mData(new T[1]()), mCap(1),
              mSize(0), mFront(0) { }
    // copy constructor
    queue(const queue<T>& a) : mData(new T[a.mCap]()), mCap(a.mCap),
                               mSize(a.mSize), mFront(a.mFront) {
      for (size_t i = 0; i < a.mCap; i++) {
        mData[i] = a.mData[i];
      }
    }
    // copy assignment operator
    queue<T>& operator=(queue<T> other) {
      using std::swap;
      swap(mSize, other.mSize);
      swap(mCap, other.mCap);
      swap(mData, other.mData);
      swap(mFront, other.mFront);
      return *this;
    }
    ~queue() {
      delete [] mData;
    }
};
```

front(), back(), pop()

```
template <typename T>
class queue {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;
    size_t mFront;
  public:
    // access
    const T& front() const {
      return mData[mFront];
```

```
    }
    const T& back() const {
      return mData[(mFront + mSize - 1) % mCap];
    }
    // modify
    void pop() {
      mFront = (mFront + 1) % mCap;
      mSize--;
    }
};
```

- `back = mFront + mSize - 1`
  - Also circular (by `% mCap`)
- `pop` = move `mFront` by 1
  - Also circular
  - Also change size

push, expand

- `push` add data to `(mFront + mSize) % mCap`
  - The space just after `back()`
- Expand re-pack the `mData` so that `mFront` is 0
- `ensureCapacity` is the same

```
template <typename T>
class queue {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;
    size_t mFront;
    void expand(size_t capacity) {
      T *arr = new T[capacity]();
      for (size_t i = 0; i < mSize; i++) {
        arr[i] = mData[(mFront + i) % mCap];
      }
      delete [] mData;
      mData = arr;
      mCap = capacity;
      mFront = 0;
    }
    void ensureCapacity(size_t capacity) {
      if (capacity > mCap) {
        size_t s = (capacity > 2 * mCap) ? capacity : 2 * mCap;
        expand(s);
      }
    }
  public:
    void push(const T& element) {
      ensureCapacity(mSize+1);
      mData[(mFront + mSize) % mCap] = element;
      mSize++;
    }
};
```

Analysis

- All access, modification is fast (constant time)

- Space is re-used
  - It is not shrunk when `mSize` reduce
  - Space is not more than double of maximum `mSize` during its lifetime

Queue Exercise

- We implement circular queue by maintain `mFront` and use circular logic (`% mCap`) to calculate the position of back of the queue
  - Can we maintain `mBack` instead?
  - Can we maintain both `mFront` and `mBack` but not `mSize`?
- How about `mCap`, if we know `mFront`, `mSize`, `mBack`, can we calculate `mCap`?

`mFront`	`mSize`	`mBack`	`front()`	`back()`	`size()`
YES	YES	No	`v[mFront]`	`v[(mFront + mSize) % mCap]`	`mSize()`
No	YES	YES	`????`	`v[mBack]`	`mSize`
YES	No	YES	`v[mFront]`	`v[mBack]`	`????`

Answer

- Use `front() = v[(mCap + mBack - mSize) % mCap]`
- Use `size() = (mCap + mBack - mFront) % mCap`, but this will break when `mBack == mFront` because it can't determine whether the queue is empty or full.
- Impossible, `mCap` can be scaled to any size larger than the current elements.

Now, meet deque

- Can you modify queue to include
  - `push_front()`, add to the front of the queue
  - `pop_back()`, remove from back of the queue
- All operation should still be constant time

Answer

```
void push_front(const T& element) {
  ensureCapacity(mSize+1);
  mFront = (mCap + mFront - 1) % mCap;
  mData[mFront] = element;
  mSize++;
}
void pop_back() {
  mSize--;
}
```

Lecture 12

Complexity Analysis

How fast is our code?

Overview

- Introducing a measurement of efficiency of a program
- Why we need measurement
- How we measure
  - How to analyze our code to get a measurement
- What is measurement
  - Asymtotic Notation

Key Idea

- We need a useful way to describe effiency of our code
  - Useful = able to be easily use to predict how much resource (time / memory) that our program will need
  - Useful = not overly complex in analysis
  - Need to balance between usefulness and complexity
- Ultimately, we introduce a class of efficiency that says how our code use resource with respect to size of data
  - Focus on on growth of resource usage

Preview

```
int find_max(vector<int> v) {
  int m = v[0];
  for (size_t i = 0; i < v.size(); i++)
    if (v[i] > m)
      m = v[i];
  return m;
}
```

- This code takes time directly proportional to the size of the data
  - Size N takes time T
  - Size $5N$ should take time $5T$

```
int count_pair_sum(vector<int> v, int k) {
  int count = 0;
  for (size_t i = 0; i < v.size(); i++)
    for (size_t j = 0; j < v.size(); j++)
      if (i != j && v[i] + v[j] == k)
        count++;
  return count/2;
}
```

- This code takes time directly proportional to the square of the size of the data
  - Size N takes time T
  - Size $5N$ should take time $25T$

Why don't we use real world clock?

- Ultimately, we want to know how long our program takes to do each operation
  - Use in design, how much resource we need
  - Help use choose appropriate data structure
- Real world clock measurement is possible but has many drawback
  - System dependency
  - Too complex (we have to build our system)
  - Too specific

Dependency of System

- Same program in different system = different time
  - CPU
  - RAM
  - Compiler
  - Operating parameter
    - Heat? Other running program?

How to measure

- Counting instruction
  - Depend on code only
- Dependency on size of data
  - Focus on large data

```
int count_pair_sum(vector<int> v, int k) {
  int count = 0;                          // 1
  for (size_t i = 0; i < v.size(); i++)   // 2 + n + n  ( n = v.size() )
    for (size_t j = 0; j < v.size(); j++) // (2 + n + n) * n
      if (i != j && v[i] + v[j] == k)     // 2 * n * n
        count++;                          // 1 * ????  ( <= (d) )
  return count/2;                         // 1
```

```
}

// Total = 1 + 2+n+n + (2+n+n)n + 2n^2 + n^2 + 1
//       = 4 + 4n + 5n^2
```

Time per instruction

- In reality, different CPU instructions use different time
- Same instruction but different CPU also use different number of cycle
- However, we just ignore it
  - For now

Focusing on large data

- In many cases, the size of the data we are working with will affect the time our code use
- Large data usually mean longer time
- What matter is when the data is large

Growth rate simplifies analysis

```
int count_pair_sum(vector<int> v, int k) {
  int count = 0;                                  //     i=0: 1 + n-0 + n-1
  for (size_t i = 0; i < v.size(); i++)           //     i=1: 1 + n-1 + n-2
    for (size_t j = i+1; j < v.size(); j++)       // <-- i=2: 1 + n-2 + n-3
      if (v[i] + v[j] == k)                       //     . . .
        count++;                                  //     i=n-1: 1 + 1 + 0
  return count;                                   //     sum = Σ(1+2n-2i-1) = n^2 + n
}

// Total = 1 + 2+n+n + n^2+n + n^2 + n^2 + 1
//       = 4 + 3n + 3n^2
```

Small Detail

n	$5n^2+4n+4$		$3n^2+3n+4$		n^2	
	raw	growth	raw	growth	raw	growth
10	544		334		100	
20	2084	3.830 88	1264	3.784 43	400	4
40	8164	3.917 47	4924	3.895 57	1600	4
80	32 324	3.959 33	19 444	3.948 82	6400	4
160	128 644	3.979 83	77 284	3.9747	25 600	4
320	513 284	3.989 96	308 164	3.987 42	102 400	4
640	2 050 564	3.994 99	1 230 724	3.993 73	409 600	4
1280	8 197 124	3.9975	4 919 044	3.996 87	1 638 400	4
2560	32 778 244	3.998 75	19 668 484	3.998 44	6 553 600	4
5120	1.31×10^8	3.999 37	78 658 564	3.999 22	26 214 400	4
10 240	5.24×10^8	3.999 69	3.15×10^8	3.999 61	1.05×10^8	4
20 480	2.1×10^9	3.999 84	1.26×10^9	3.9998	4.19×10^8	4

When counting instruction, it is usually OK to focus on most executed line

Measurement by Growth Rate

What? **Why?**

- Growth rate = how much resource usage growth with respect to change of input
  - Resource usage = number of instruction used
  - Input = size of data
- Emphasizes long term trend
- System independent
  - The result can be used to predict behavior on any system
- Focus on change of resource usage with respect to size of input
- Can regard small detail
  - Simple to calculate
  - Applicable in real world

Asymtotic Notation

Classification of growth rate

Overview

- Formally, it is a set of function having the growth rate related to something
- The definition focus on growth of the function while disregard small detail
- Also provide some workaround on dependency of value of input

What is?

- A kind notation written as $O(f(n))$
  - O can be one of $\mathcal{O}, \Theta, \Omega, o, \omega$
  - $f(n)$ is some expression
- Example $\mathcal{O}(n)$ or $\Theta(n^2+3)$ or $\omega(n^2 \log(n))$
- Usage
  - "This code is $\mathcal{O}(n)$" (read as Big-Oh of n)
  - "This function takes time in $\Theta(\log(n))$" (read as Big-theta of log n)
  - "Time complexity of this program is $\mathcal{O}(n^2)$"

Meaning

- "A is $\Theta(f(x))$" means the growth rate of A is equal to the growth rate of $f(x)$
- "B is $\mathcal{O}(f(x))$" means the growth rate of B is less than or equal to the growth rate of $f(x)$
- For Ω, o, ω, (Big-Omega, little-oh, little-omega), we won't use it for now but the meaning is similar (which are more than or equal, less than, more than, respectively)
- Conventionally, we usually use N for the size of the data

Usage

- Let $f(n)$ be the number of instruction needed by code A when the size of data is n
- We will calculate asymtotic notation that $f(n)$ is a member of
  - Find $g(n)$ such that $f(n)$ is $\mathcal{O}(g(n))$ or $\Theta(g(n))$ (or $\Omega(g(n))$ or ...)
- Let's say we have analyze that $f(n)$ is $\mathcal{O}(g(n))$
  - We now understand that the growth rate of instruction required by n grows slower or the same as how $g(n)$ grow

Comparing growth rate of f(n) and g(n)

- The relation of growth rate of $f(n)$ and $g(n)$ depends on the value of $f(n)/g(n)$ when $n \to \infty$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \text{if } f(n) \text{ grows slower than } g(n) \\ c & \text{if } f(n) \text{ grows similarly to } g(n) \\ \infty & \text{if } f(n) \text{ grows faster than } g(n) \end{cases}$$

Example

- $f(n) = 4 + 3n + 4n^2$
- $g(n) = n^2$

$$\lim_{n\to\infty}\frac{4n^2+3n+4}{n^2}=\lim_{n\to\infty}\left(4+\frac{3}{n}+\frac{4}{n^2}\right)=4$$

- Hence $f(n)$ grows similarly to $g(n)$
  - Therefore, $f(n)=\Theta(n^2)$

Another Example

- $f(n)=0.00005n^2$
- $g(n)=100000n$

$$\lim_{n\to\infty}\frac{0.00005n^2}{100000n}=\lim_{n\to\infty}10^{-10}n=\infty$$

- Hence $f(n)$ grows faster than $g(n)$ (also means $g(n)$ grows slower than $f(n)$)
  - Therefore $g(n)=\mathcal{O}(n^2)$
  - and $f(n)=\Omega(10000n)$

Dependency on the value of input

- Consider `vector::insert(iterator it, T value)`
- The time it takes depends on both the size of the vector and the value of it
  - Larger size $\longrightarrow$ more time
  - Closer to `end()` $\longrightarrow$ less time

Big-O describes upper bound

- `vector::insert` is $\mathcal{O}(n)$
  - Its growth rate does not exceed n
  - There are cases that it may grow less than n (insert at `end()`)
- This is very useful in real world
  - Knowing maximum load
  - Not overly complex in analysis

Big-O Example

- Find is $\mathcal{O}(n)$
  - At worse, it can't find value and `a` & `b` points to `begin()` & `end()`
    - This case, find growth rate is n
  - At best, it always find value at the first position (`a`)
    - This case, find growth as 1

```
bool find(iterator a, iterator b, T value) {
  while (a < b) {
    if (*a == value)
      return true;
    a++;
  }
  return false;
}
```

L'Hôpital's Rule

$$\lim_{n\to\infty}\frac{f(x)}{g(x)}=\lim_{n\to\infty}\frac{f'(x)}{g'(x)}$$

If $f(x), g(x)$ must be diffable, $g(x)$ is non-zero, $\lim_{n\to\infty} f'/g'$ exists

- Can help
- $F(n)=\log n$
- $g(n)=n^{0.5}$

$$\begin{aligned}\lim_{n\to\infty}\frac{\log(n)}{\sqrt{n}} &= \lim_{n\to\infty}\frac{\ln(n)}{\ln(10)\sqrt{n}}\\ &= \frac{1}{\ln(10)}\lim_{n\to\infty}\frac{\ln(n)}{\sqrt{n}}\\ &= \frac{1}{\ln(10)}\lim_{n\to\infty}\frac{\frac{1}{n}}{\frac{1}{2\sqrt{n}}}\\ &= \frac{1}{\ln(10)}\lim_{n\to\infty}\frac{2}{\sqrt{n}}\\ &= 0\end{aligned}$$

Exercise

- $f(n) = \lg^c n$
- $g(n) = n^k$
- We know that $c > 0, k > 0$
- Does $f(n)$ grow slower than $g(n)$?

Will do this part later

Big Theta

Big-Theta is tight bound

- `std::count` always go through entire array
- Regardless of the value in the array, it always perform `if (*first == value)`

```
size_t count(iterator first, iterator last, const T& value) {
  size_t ret = 0;                    // 1
  for (; first != last; ++first) {   // n + n
    if (*first == value)             // 1 * n
      ret++;                         // 1 * ???? ( <= (d) )
  }
  return ret;                        // 1
}

// Total = 1 + n+n + 1*n + 1*n + 1
//       = 2 + 4n
```

More Example

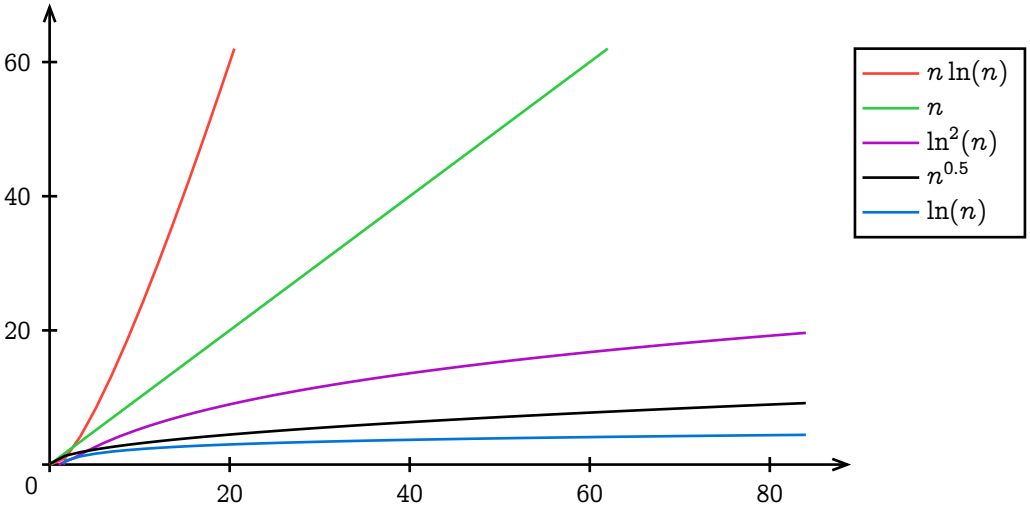
$\Theta(1)$	$\Theta(n)$	$\Theta(n^2)$
$f(n) = 5$ $f(n) = 0$ $f(n) = c$	$f(n) = n$ $f(n) = n + 3$ $f(n) = \frac{n}{1200} + 86$ $f(n) = 40000000n$	$f(n) = n^2$ $f(n) = \binom{n}{2} = \frac{n(n-1)}{2}$ $f(n) = 400n^2 + an + b$

Observation: multiplicative and addition constants in $f(n)$ can usually be ignored, since it will be disragard by lim, Degree cannot.

Well known growth rate class

$\Theta(1)$	Constant
$\Theta(\log(n))$	Logarithm
$\Theta(\log^c(n)), c \geq 1$	Polylogarithm
$\Theta(n^a), 0 < a < 1$	Sublinear

$\Theta(n)$	Linear
$\Theta(n \log(n))$	Linearithmic
$\Theta(n^2)$	Quadratic
$\Theta(n^c), c \geq 1$	Polynomial
$\Theta(c^n), c > 1$	Exponential
$\Theta(n!)$	Factorial



$\ln^2(n)$ grows slower than $n^{0.5}$. At around $n = 56000$, $\ln^2(n)$ is less than $n^{0.5}$.
x^4 grows slower than 2^n. At around $n = 16$, x^4 is less than 2^n.

Beware

- It is wrong to say that `vector::insert` is $\Theta(n)$
  - Because there is a case that it grows slower than n
- It is wrong to say that `vector::push_back` is $\Theta(n)$
  - Because there is a case that it grows slower than n
- It is ok to say that `std::count` is $\mathcal{O}(n)$
  - Because while it always grows as n, it does not grow faster than n
  - $\mathcal{O}(n)$ is upper bound
  - But it is better to say that `std::count` is $\Theta(n)$

How to analyze using asymptotic notation

1. Write a code
2. Calculate the function $f(n)$ that counts the number of instruction of the code when the data is size n
   - Usually, just focus on most executed line
3. Find $g(n)$ and a notation X such that $f(n)$ is $X(g(n))$
   - It's either Big-O or Big-Theta. If there is a case that it can grow slower than $G(n)$, use Big-O

Another Example

- Let's analyze `vector::push_back`

```
iterator insert(iterator it, const T& element) {
  size_t pos = it - begin();
  ensureCapacity(mSize + 1);
  for (size_t i = mSize; i > pos; i--) {
    mData[i] = mData[i-1];  // <-----  Most executed line: 1 (b)
  }
  mData[pos] = element;
  mSize++;
```

```
  return begin()+pos;
}
void push_back(const T& element) {
  insert(end(), element);
}

void expand(size_t capacity) {
  T *arr = new T[capacity]();
  for (size_t i = 0; i < mSize; i++) {
    arr[i] = mData[i];  // <-----  Most executed line: 0, 0, 0, 0, 2, 4, 8, 16 (a)
  }
  delete [] mData;
  mData = arr;
  mCap = capacity;
}

void ensureCapacity(size_t capacity) {
  if (capacity > mCap) {
    size_t s = (capacity > 2 * mCap) ? capacity : 2 * mCap;
    expand(s);
  }
}

// (a) can be 0 to n
// (b) can also be 0 to n
// Best case 0 + 0
// Worst case n + n
// vector::push_back is O(n)
```

Another Definition for $\mathcal{O}$ and Θ

- Using set builder notation
- $\mathcal{O}(g(n)) = \{f(n) \mid \text{there exists } c > 0 \text{ and } n_0 \geq 0 \text{ such that } f(n) \leq cg(n) \text{ for } n \geq n_0\}$
- $\Theta(g(n)) = \{f(n) \mid \text{there exists } c_1, c_2 > 0 \text{ and } n_0 \geq 0 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for } n \geq n_0\}$
- The result is the same as definition using lim

Summary

- We use Asymtotic Notation to describe efficiency of a program
  - Measure instruction count instead of time
  - Focus on growth rate of instruction count
- Find most frequently executed line in the code and count it
- Maps to Big-Theta if we have tight bound
- Use Big-O if we have upper bound

Example

Showing $\log_2(n!)$ is $\Theta(n \log_2(n))$

- Will use set definition of Big-Theta
  $\{f(n) \mid \text{there exists } c_1, c_2 > 0 \text{ and } n_0 \geq 0 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for } n \geq n_0\}$
- Need to find c_1, c_2 and n_0

Finding c_1 and c_2

$$n! = n \times (n-1) \times (n-2) \times (n-3) \times \cdots \times 1$$

$$n! \leq n \times n \times n \times n \times \cdots \times n$$

$\log n! \leq \log n^n$ $\quad$ $\boxed{\log n^n = n \log n \text{ when } n \geq 1}$

Found $c_2 = 1$ for upper bound $\boxed{\log n! \leq n \log n \text{ when } n \geq 1}$

$$n! = n \times (n-1) \times \cdots \times \lfloor \tfrac{n}{2} \rfloor \times (\lfloor \tfrac{n}{2} \rfloor - 1) \times \cdots \times 1$$
$$n! \geq \lfloor \tfrac{n}{2} \rfloor \times \lfloor \tfrac{n}{2} \rfloor \times \cdots \times \lfloor \tfrac{n}{2} \rfloor \times 1 \times \cdots \times 1$$
$$n! \geq (\tfrac{n}{2})^{n/2} \qquad \boxed{\log(\tfrac{n}{2})^{n/2} = \tfrac{n}{2}\log n - \tfrac{n}{2}}$$
$$\log n! \geq \log(\tfrac{n}{2})^{n/2} \qquad \boxed{0.5n \log n - 0.5n \geq 0.4n \log n \text{ when } n \geq 32}$$

Found $c_1 = 0.4$ for lower bound $\boxed{\log n! \geq 0.4n \log n \text{ when } n \geq 32}$

$\boxed{0.4n \log n \leq \log n! \leq n \log n \text{ when } n \geq 32, c_1 = 0.4, c_2 = 1, n_0 = 32}$

$\implies \boxed{\log n! \text{ is } \Theta(n \log n) \text{ and } n \log n \text{ is } \Theta(\log n!)}$

Lecture 13

Priority Queue Simple Implementation

Featuring Binary Heap

Overview

- Simple implementation of `priority_queue`
- Quick intro to Graph and Tree
- Binary Heap
- `priority_queue` with Binary Heap

priority_queue

- Queue by value
- Max-in-First-Out

```
int main() {
  priority_queue<int> pq;
  pq.push(4);
  pq.push(20);
  pq.push(3);

  while (pq.empty() == false) {
    cout << pq.top() << endl;
    pq.pop();
  }
}
```

Vo.1, priority_queue by vector

- Use vector to store data
- `push` = simply `push_back`
- `top`, `pop` = find the max value and `return` / `erase`
- `max_element` returns iterator to max element

```
namespace CP {
  template <typename T>
  class priority_queue {
    protected:
      std::vector<T> v;
    public:
      bool empty(); { return v.empty(); }
```

```
    bool size(); { return v.size(); }

    void push(const T &e) { v.push_back(e); }

    T top() { return *std::max_element(v.begin(), v.end()); }

    void pop() { v.erase(std::max_element(v.begin(), end())); }
  };
}
```

max_element

```
iterator max_element(iterator first, iterator last) {
  if (first == last) return last;
  iterator largest = first;
  ++first;
  for (; first != last; ++first)
    if (*largest < *first)
      largest = first;
  return largest;
}
```

Vo.1 complexities

`push`	$\mathcal{O}(1)$ *amortized
`top()`	$\Theta(n)$
`pop()`	$\Theta(n)$

Vo.2 faster pop, top (and push??)

- vo.1 has many drawbacks
  - Consecutive call of `top` is slow (it shouldn't)
  - Both pop and `top` works almost the same
- vo.2 focus on slower push while keeps pop, `top` fast
- Make the array sorted
  - Max will be at the back
  - Fast pop, `top`

```
namespace CP {
  template <typename T>
  class priority_queue {
    protected:
      std::vector<T> v;
    public:
      bool empty(); { return v.empty(); }
      bool size(); { return v.size(); }

      T& top() { return v[v.size()-1]; }

      void pop() { v.erase(v.end()-1); }

      void push(const T &e) {
        // do something
      }
  };
}
```

v0.2 push

- Maintain that the vector is sorted at every push

```
void push(const T& e) {
  v.push_back(e);  // <-- O(1)
  sort(v.begin(), v.end());  // <-- O(n log(n))
}
```

```
void push(const T& e) {
  auto it = v.begin();
  while (it < v.end() && *it <= e)  // <-- O(n)
    it++;
  v.insert(it,e);  // <-- O(n)
}  // Total of this program is Θ(n)
```

```
void push(const T& e) {  // upper_bound is O(log(n))
  v.insert(std::upper_bound(v.begin(), v.end(), e), e);  // <-- O(n)
}
```

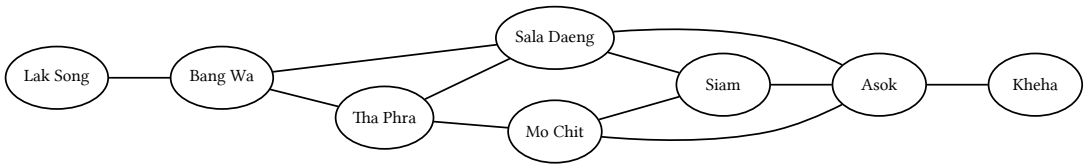
Which one is better?

- v0.1 fast `push`
- v0.2 fast `pop`, `top`
- Depends on which operation we use most often
- The real version works like v0.2, we maintain some rules of the data that is stored in the vector such that
  - We know where max is (for fast `pop`)
  - Much faster `push`, a little bit slower `pop`
  - Use structure called Binary Heap

Graph & Tree

Graph

- Discrete Math Graph
- A math model that describe entities and connectivity between them



Graph Model

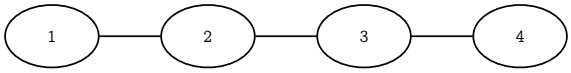
- Graph consists of two things
  - Nodes (vertex, vertices) are things we want to connect
  - Edges are pairs, each pair is a connectivity between two nodes
- Graph $G = (V, E)$ where V is a set of nodes and E is a set of edges

$V = \{$"Mo Chit", "Siam", "Asok", "Sala Daeng", "Tha Phra", "Lak Song", "Bang Wa", "Kheha"$\}$
$E =$
$\{$("Mo Chit", "Asok"), ("Mo Chit", "Siam"), ("Tha Phra", "Mo Chit"), ("Sala Daeng", "Tha Phra"), ...$\}$

Tree

- A special kind of graph
  - Has N nodes and $N - 1$ edges

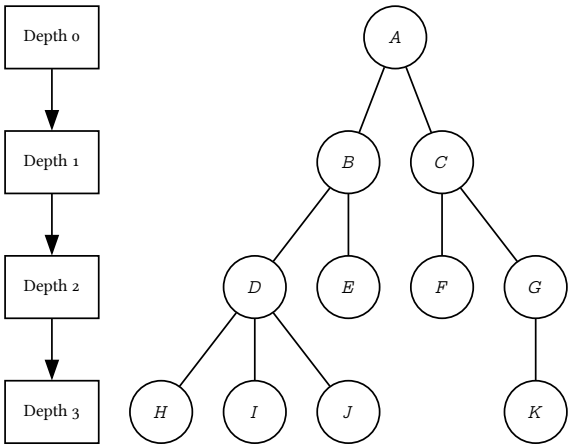
- Every nodes must be connected (we can start from any node and can walk through edge to reach any node)



$$V = \{1, 2, 3, 4\}$$
$$E = \{(1, 2), (2, 3), (3, 4)\}$$

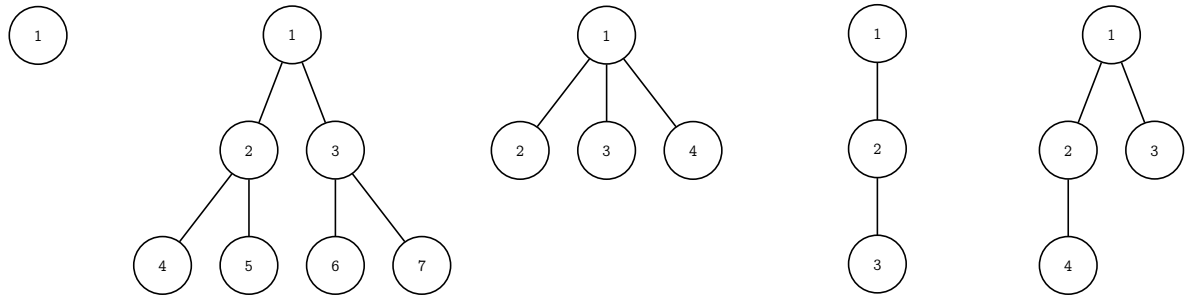
Rooted Tree

- Tree where one node is defined as root
- For an edge in a rooted tree, a node that is closer to the root is called parent while the other node is called child
- Ancestor of node A = parent of parent ... of A
- Descendant of node A = child of child ... of A
- Root is usually drawn at the top and is considered as the starting point
  - Root is at level 0 (depth 0)
  - Children of root are drawn at the same level at level 1 (depth 1)
  - Children of children of root are at level 2 (depth 2)



Complete Binary Tree

- Binary Tree = a tree that every node has at most 2 children
- Complete tree = the tree must be filled with every possible node at every level (except the deepest level which must be filled as far to the left as possible)
  - Blank tree is considered a complete binary tree

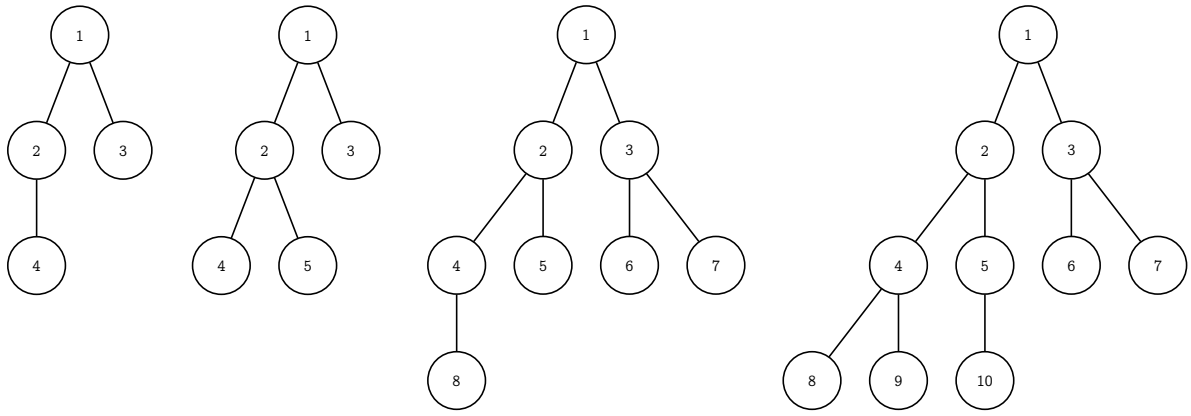


OK OK Not binary Not complete (at depth 1) OK

Exercise

- Draw a Complete Binary Tree that has 4, 5, 8, 10 nodes

Hint: The answer is unique (There are exactly 1 way to draw a complete binary tree of size k)

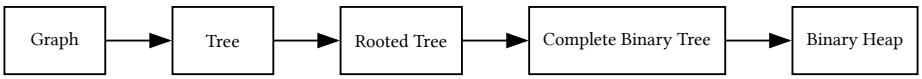


Special Property of a Complete Binary Tree

- There is exactly one way to go from any node to any node
- Maximum depth is $\log_2 n$ where n is the number of nodes
  - Because we require completeness, and we have 2 possible children

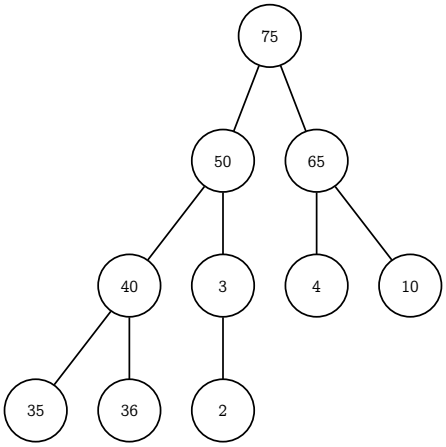
Binary Heap

Use Complete Binary Tree to make `priority_queue`



Binary Heap

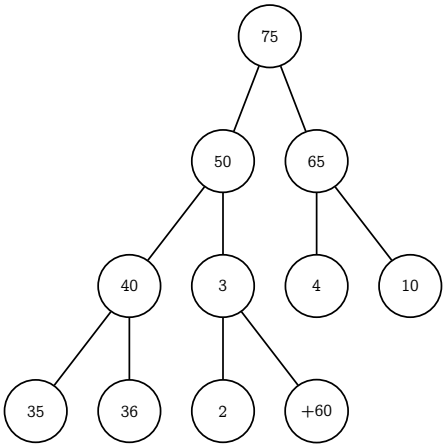
- We use Complete Binary Tree to store data
  - A value is stored at the node
- When data is modified (via `push` or `pop`), we must maintain these rules
  1. Tree must always be Complete Binary Tree
  2. For any node, its value must be greater or equal that of its children



Adding data to Binary Heap

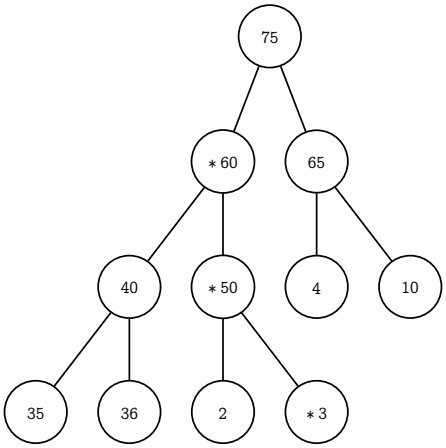
- Maintain Binary Heapness
  - structure of data
  - value of data
- Structure rules says where the new nodes should be
  - Next to right-most child of deepest level
  - But if we put new data there, the value rules might be broken
    - Fix it

`push(60)`



Fix from adding a new node

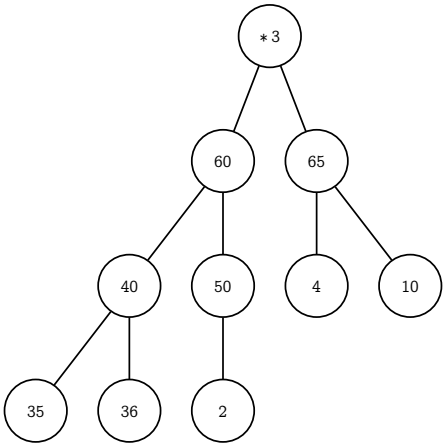
- Fix:
  - Check where we just add a node, if value rules is broken, swap with parent
  - After swap, re-check with new parent
  - Kepp doing until correct or at root



See that: after each swap, the swapped nodes does not violate with its new children

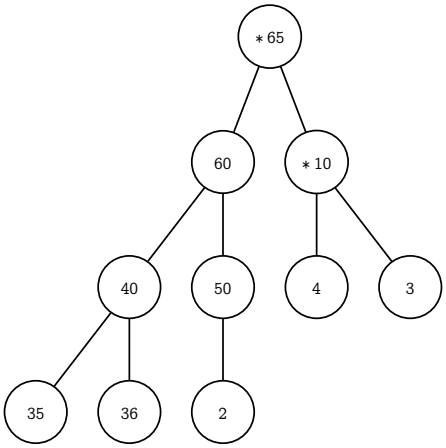
Delete maximum data

- Similar to `push`, we will try to maintain structure first
- `delete` will remove root, find something to replace
  - Use the lowest, right-most node
- Value rules might be broken
  - Fix it



Fix from deleting root node

- Fix:
  - Start at replaced root, if value rules is broken, swap with maximum child
  - After swap, re-check with new children
    - Beware! There is a case where we might have only one child
  - Keep doing until correct or has no children



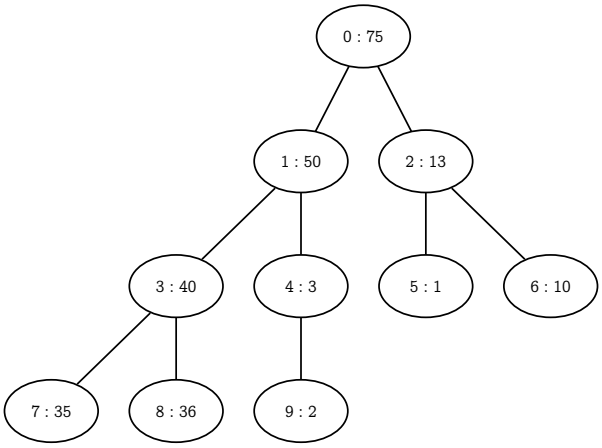
Analysis

- How fast is `push`, `pop`
- `push`
  - Add to vector is $\mathcal{O}(1)$ amortized
  - Fixing value rules is $\mathcal{O}(h)$ where h is the maximum number of depth of the tree (we call this value tree height)
  - Notice that tree height is $\mathcal{O}(\log(n))$
- `pop`
  - Fixing value rules is $\mathcal{O}(h)$ where h is the maximum number of tree height

Push + Fix Up

How to store a tree?

- Use dynamic array
  - Each node can be labelled from `0` to `n-1`
- Root is at `0`
- Left child of node `i` is at `(i*2)+1`
- Right child of node `i` is at `(i*2)+2`
- Parent of node `i` is at `(i-1)/2`




```
                0   1   2   3   4   5   6   7   8   9
Dynamic array: [75, 50, 13, 40,  3,  1, 10, 35, 36,  2]
```

Layout

```
template <typename T, typename Comp = std::less<T>>
class priority_queue {
  protected:
    T *mData;
    size_t mCap;
    size_t mSize;
    Comp mLess;
    void expand(size_t capacity) {}  // Fix value rules
    void fixUp(size_t idx) {}
    void fixDown(size_t idx) {}
  public:
    // constructor
    priority_queue(priority_queue<T,Comp>& a);
    priority_queue(const Comp& c = Comp());
    priority_queue<T, Comp>& operator=(priority_queue<T,Comp> other);
    ~priority_queue();
    // capacity function
    bool empty() const;
    size_t size() const;
    // access
    const T& top();
    // modifier
    void push(const T& element);
    void pop();
};
```

Constructor

- Using list initialize
- See that `mData` is dynamic array in the same way as vector
- `mLess` is something that is just either copied or default initialize
  - Will talk about it later

```
priority_queue(const Comp& c = Comp()):
  mData( new T[1]() ),
  mCap( 1 ),
  mSize( 0 ),
  mLess( c )
{ }

priority_queue(priority_queue<T, Comp>& a) :
  mData(new T[a.mCap]()),
  mCap(a.mCap),
  mSize(a.mSize),
  mLess(a.mLess)
{
  for (size_t i = 0; i < a.mCap; i++) {
    mData[i] = a.mData[i];
  }
}
```

Destructor and Copy Assignment Operator

- Using standard copy-and-swap idiom

```
~priority_queue() {
  delete [] mData;
```

```
}

priority_queue<T,Comp>& operator=(priority_queue<T,Comp> other) {
  using std::swap;
  swap(mSize,other.mSize);
  swap(mCap,other.mCap);
  swap(mData,other.mData);
  swap(mLess,other.mLess);
  return *this;
}
```

Push

- See that the right-most child of the deepest level is at `mData[mSize-1]` and the new node should be at `mData[mSize]`
- We do the same thing as vector's `push_back`
- Then fix the value rule

```
void expand(size_t capacity) {  // Same as CP::vector
  T *arr = new T[capacity]();
  for (size_t i = 0; i < mSize; i++) {
    arr[i] = mData[i];
  }
  delete [] mData;
  mData = arr;
  mCap = capacity;
}

void push(const T& element) {
  if (mSize + 1 > mCap)
    expand(mCap * 2);
  mData[mSize] = element;
  mSize++;
  fixUp(mSize-1);
}
```

Fix Up

- Instead of actual swap, we perform `insert` and `find` appropriate position at the same time

```
void fixUp(size_t idx) {
  T tmp = mData[idx];
  while (idx > 0) {
    size_t p = (idx - 1) / 2;
    if ( tmp < mData[p] ) break;
    mData[idx] = mData[p]
    idx = p;
  }
  mData[idx] = tmp;
}
```

```
mData  p = /, idx = *, tmp = 60
   0   1   2   3   4   5   6   7   8   9  10
[ 75, 50, 13, 40, /3,  1, 10, 35, 36,  2,*60]

[ 75,/50, 13, 40, *3,  1, 10, 35, 36,  2,  3]

[/75,*50, 13, 40, 50,  1, 10, 35, 36,  2,  3]
```

```
tmp = 60 < 75 = mData[p] => break;mData[idx] = tmp;
[ 75, 60, 13, 40, 50,  1, 10, 35, 36,  2,  3]
```

mLess

- `priority_queue` allows a custom comparator
- Custom comparator X
  - We must be able to $X(a, b)$ where X will comapre a and b and `return true` only when a is less than b
  - X is a variable that implement `operator()`

- Initialize at constructor as variable `mLess` to be of type `Comp` in template
- Any comparison of our data (type `T`) must be done by `mLess`

```
void fixUp(size_t idx) {
  T tmp = mData[idx];
  while (idx > 0) {
    size_t p = (idx - 1) / 2;
    if ( mLess(tmp, mData[p]) ) break;
    mData[idx] = mData[p]
    idx = p;
  }
  mData[idx] = tmp;
}
```

Pop + FixDown

Pop

```
void pop() {
  mData[0] = mData[mSize-1];
  mSize--;
  fixDown(0);
}

void fixDown(size_t idx) {
  T tmp = mData[idx];
  size_t c;
  while ((c = 2 * idx + 1) < mSize) {
    if ( c + 1 < mSize && mLess(mData[c], mData[c+1]) ) c++;
    if ( mLess(mData[c],tmp) ) break;
    mData[idx] = mData[c];
    idx = c;
  }
  mData[idx] = tmp;
}
```

- `while` loop checks if we have at least one child
- `c` is the index of highest value child
  - Must consider the case where we have only one child
- Exercise: read the rest yourself

Fast Construction

Construct PQ from n data

```
priority_queue(std::vector<T> &v, const Comp& c = Comp()) :
    mData( new T[v.size()]() ), mCap( v.size() ), mSize( 0 ), mLess( c ) {
  for (size_t i = 0; i < mSize; i++) push(v[i]);  // n times and each log(n)
}
```

$$\begin{aligned}\text{Total} &\leq \lfloor \log_2 1 \rfloor + \lfloor \log_2 2 \rfloor + \ldots + \lfloor \log_2 n \rfloor \\ &\leq \lfloor \log_2(1 \times 2 \times 3 \times \cdots \times n) \rfloor = \lfloor \log_2 n! \rfloor\end{aligned}$$

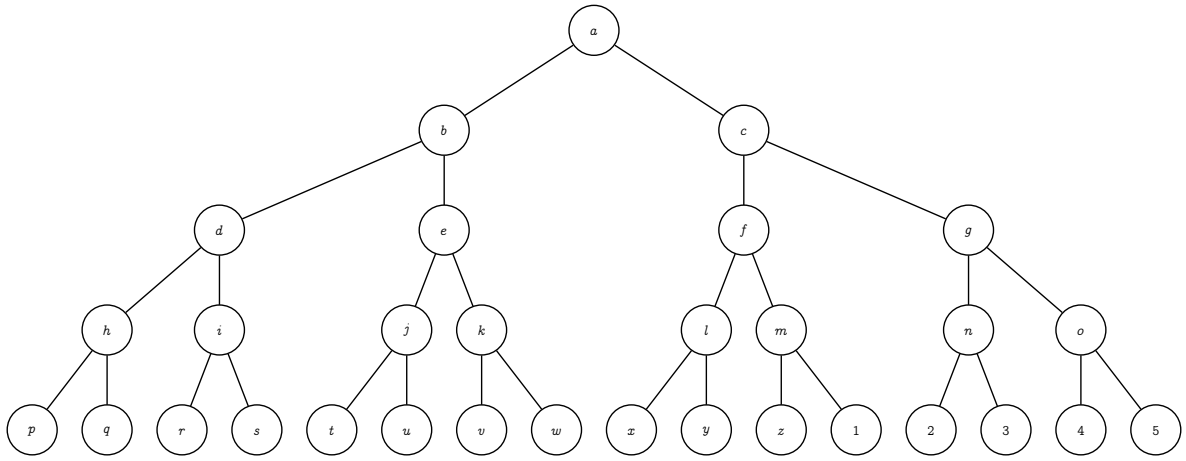
$\lfloor \log_2 n! \rfloor$ is $\mathcal{O}(n \log n)$

Better Method

```
priority_queue(std::vector<T> &v, const Comp& c = Comp()) :
    mData( new T[v.size()]() ), mCap( v.size() ), mSize( 0 ), mLess( c )
{
  for (size_t i = 0; i < mSize; i++) mData[i] = v[i];
  for (int i = mSize/2-1; i > 0; i++) fixDown(i);
}
```

- Consider each node to be a binary heap
- Fix down from back to front

How fast?



- Binary Tree Property:
  - There are at most 2^k nodes at depth k
  - For a tree of height h, at depth k, fix down needed at most $h - k$ iterations

Total nodes $= 31$
Tree height $= \log_2(31) = 4$

Depth: k, Nodes: 2^k, Max fix: $h - k$
Total $= \sum_{k=0}^{h} k2^{h-k}$

Depth	Nodes	Max fix per node
0	1	4
1	2	3
2	4	2
3	8	1
4	16	0

$$\sum_{k=0}^{h} k2^{h-k} = 2^h \sum_{k=0}^{h} k2^{-k} < 2^h \underbrace{\sum_{k=0}^{\infty} k2^{-k}}_{\text{This is 2}} = 2^{h+1} = 2^{\log_2 n+1} = \mathcal{O}(n)$$

Exercise

If we use different number of children per node, say 4-ary heap, with same logic still holds.

1. How to keep data in dynamic array?

2. Write the new functions `fixUp` and `fixDown`
3. Is it faster?? Consider any n-ary heap

Will do this part later

Lecture 14

Linked List

Faster insert and erase but slower access

Overview

- Vector takes $\mathcal{O}(n)$ in `insert` / `erase` at position specified by an iterator
  - But it is very fast to access any member, $\mathcal{O}(1)$
- List gives better performance on `insert` / `erase` as $\mathcal{O}(1)$, providing that we have iterator to the position of insertion / erase
- Achieved by storing data into a node and each node use a pointer to identify the next element
  - Access to any elements is $\mathcal{O}(n)$, if we don't have an iterator to that element
- Use more memory than vector

List vs Vector

List	Vector
• Allocate each data separately ▸ Each data points to where is the next data ▸ Very fast `insert` / `erase` (just change some pointer) ▸ Very slow access because we don't know where k-th element is	• Allocate data as a consecutive block ▸ Very fast to access any element ▸ Very slow `insert` / `erase` requires every element after point of insertion

v0.1 node

- Simple object that stores a data and a link to another node
- NULL is a special value for any pointer that points to nowhere
  - Draw as a ground

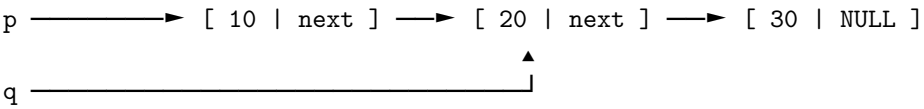
```
template <typename T>
class node {
  public:
    T data;
    node *next;
    node() :
      data( T() ), next( NULL ) { }

    node(const T& data, node* next) {
      data( T(data) ), next( next ) { }
    }
};
```

Pointer to Node

- Working with linked list needs a pointer to a node

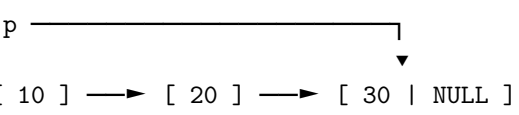
```
int main() {
  CP::node<int> *p = NULL;
  p = new CP::node<int>(10, NULL);
  CP::node<int> *q;
```

```
  q = new CP::node<int>(20, NULL);
  p->next = q;
  q->next = new CP::node<int>(30, NULL);
}
```



What is the result of?

```
p = p->next->next;  // p points to [30, NULL]
q = p;              // q points to [30, NULL]
q = q->next;        // q points to NULL
```

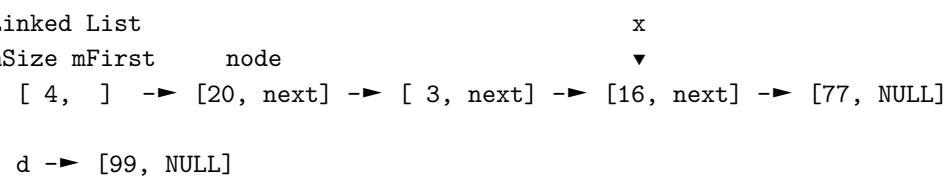


v0.1 Simple (Singly) Linked List

```
template <typename T>
class list {
  protected:
    class node {
      public:
        T data;
        node *next;
        node() :
          data( T() ), next( NULL ) { }

        node(const T& data, node* next) {
          data( T(data) ), next( next ) { }
        }
    };
  protected:
    node *mFirst;
    size_t mSize;
  public:
    list() : mFirst( NULL ), mSize( 0 ) { }
    ~list() { clear(); }
    // ... more function
};
```

- Node is a inner class

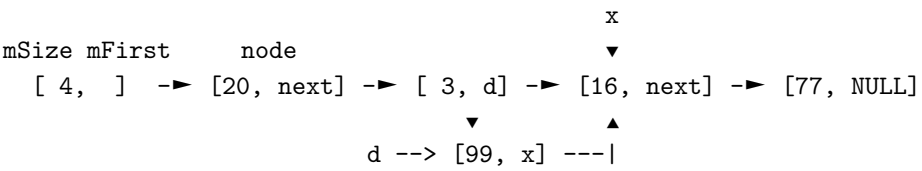
Insertion



- To insert a value 99 before value 16, let x be a pointer that point to the node of 16
  1. Create a new node d containing a data to be inserted
  2. Change pointer of a node before x (which points to x) to point to d instead

3. Make pointer of `d` points to `x`

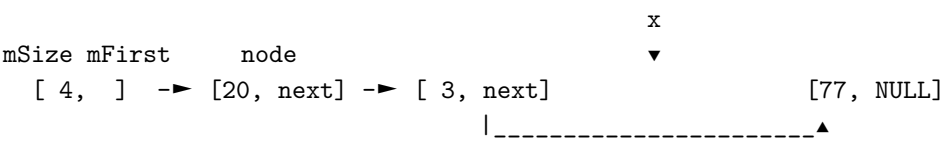
```
CP::node<int> *x = mFirst->next->next;
CP::node<int> *d = new CP::node<int>(99, NULL);
mFirst->next->next = d;
d->next = x;
```



Erase

- To delete a value of 16, in the node pointed by `x`
  1. Change pointer of a node before `x` (which points to `x`) to point to the node that `x` points to instead
  2. Don't forget to `delete x`

```
CP::node<int> *x = mFirst->next->next;
mFirst->next->next = x->next;
delete x;
```



Doubly Linked List

Problem with SLL

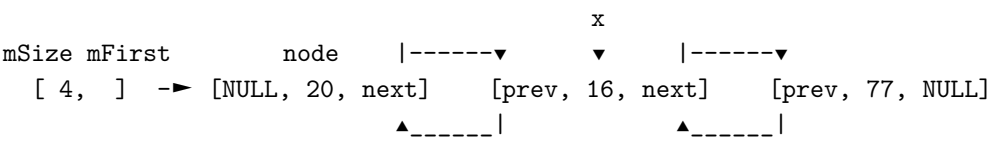
- `erase` / `insert` at iterator `X` is hard
  - If we have an iterator point to `X`, we cannot easily go to the node before `X`
  - Cannot go backward
    - Need to start from `mFirst` and move on
- Adding data to the end takes long time
  - We have to get iterator that points to the last element (which is $\mathcal{O}(n)$)

Finding node before X

```
node *p = mFirst;
while (p != NULL && p->next != x) p = p->next;
```

v0.2 Doubly Linked List

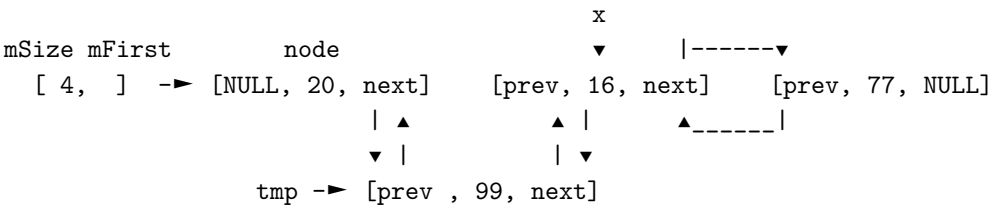
- Each node has 2 pointers
  - `next` and `prev`
- Can now move forward and backward
- Now, if `X` is a pointer to a node, we can easily go to node before `X` and then `erase` or `insert`



Insert in Doubly Linked List

```
CP::node<int> *tmp =
  new CP::node<int> (99, NULL, NULL);
tmp->next = x;
tmp->prev = x.prev;
```

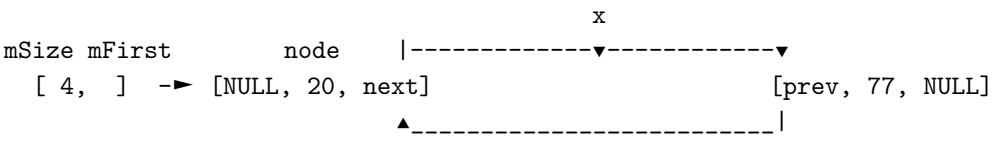
```
x->prev->next = tmp;
x->prev = tmp;
```



Erase in Doubly List

```
x->prev->next = x->next;
x->next->prev = x->prev;
delete x;
```

Note: When we refer to a pointer, `x` is the variable itself, but the value of `x` is the destination address it is pointing to. Say `x` holds a node containing `[x->prev, 16, x->next]`. In this set up, `x->prev->next` represents the next pointer inside the node before `x`. If we write `x->prev->next = x->next;`, it means that the previous node's next pointer now stores the destination of the node after `x`. In other words, it successfully skips `x` from the sequence.



Circular Linked List

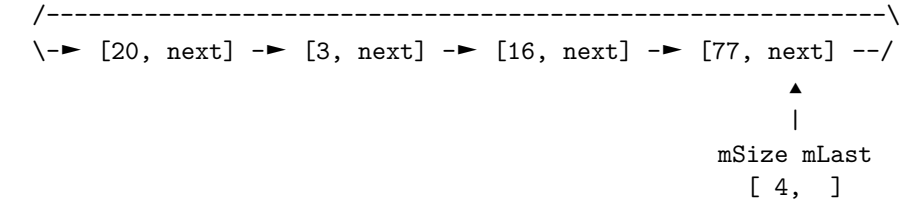
Problem Solved

- `erase` / `insert` at iterator `X` is now easy
- But, adding data at the end (`push_back`) is still hard
  - Need to get `X` to point to the last element
  - `push_back` is popular in real world
  - Right now we have only `push_back` (fast addition to the first)
- Also some minor issue about the code cleanliness
  - `insert` / `erase` the first / last node

v0.3 Circular Linked List

- Use `mLast` instead of `mFirst`
- Fast access to both first and last element

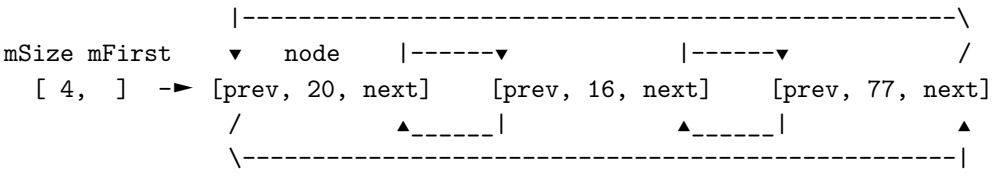
```
CP::node<int> *first = mLast->next;
```



v0.4 Circular Doubly Linked List

- Special linking to last element
  - Fast access to both first and last element
  - Can now easily insert at the end

```
CP::node<int> *last = mFirst->prev;
```



Linked List with Header

Minor Problem

- Code is not really clean because of the first element (or the last element)
- Consider `insert` at the first node and insert at the second node of Doubly Linked List
  - Assume we have a pointer to that node

```
// assume thta s points to the second node
CP::node<int> *tmp =
  new CP::node<int>(99,NULL,NULL);
tmp->next = s;
tmp->prev = s->prev;
s->prev->next = tmp;
s->prev = tmp;

// assume that f points to the first node
CP::node<int> *tmp =
  new CP::node<int>(99,NULL,NULL);
tmp->next = f;
tmp->prev = f->prev;
mFirst = tmp;  // Because first node is different
f->prev = tmp;
```

Special First Node Problem

- First node (and last node) is different from other nodes in both singly or doubly linked list
- For circular singly and circular doubly, each node look the same but we also have to adjust `mFirst` (or `mLast`)
- This affects both `insert` and `erase`
- Also need to deal when `mFirst` is `NULL` (when `mSize == 0`)

```
// circular doubly linked list
void push_front(const T& e) {
  if (mSize == 0) {
    mFirst = new node(e, NULL, NULL);
    mFirst->next = mFirst;
    mFirst->prev = mFirst;
  } else {
    node* tmp =
      new node(e, mFirst->prev, mFirst);
    mFirst->prev->next = tmp;
    mFirst->prev = tmp;
    mFirst = tmp;
  }
}
```

Another Example

- Remove for circular doubly linked list
- Remove is to find and then `erase`

```
// circular doubly linked list
void remove(const &T e) {
```

```
  node *p = mFirst;
  for (size_t i = 0; i < mSize; i++, p = p->next) {
    if (p->data == e) {
      p->next->prev = p->prev;
      p->prev->next = p->next;
      if (p == mFirst) {
        mFirst = p->next;
      }
      delete p;
      mSize--;
      break;
    }
  }
  if (mSize == 0) mFirst = NULL;
}
```

Linked List with Header

- Add a special node that will not be used to stored data

Simpler Code with Header

```
void push_front(const T& e) {
  node* f = mFirst->next;
  node* tmp = new node(e, f, mFirst);
  mFirst->next = tmp;
  f->prev = tmp;
}

void remove(const T& e) {
  node *p = mFirst->next;
  while (p != mHeader && p->data != e)
    p = p->next;
  if (p != mHeader) {
    p->next->prev = p->prev;
    p->prev->next = p->next;
    mSize--;
  }
}
```

- Header simplifies code
  - Because `mFirst` always points to the header (`mFirst` never is `NULL`)

Variant Summary

- Circular makes accessing first and last element fast
- Doubly makes accessing previous element fast
  - Also making `erase` at node `p` easy if we have pointer to `p`
  - Need more space for `prev` pointer
- Header makes code simpler
  - Need more space for header node

Final Version

- `CP::list` is "circular doubly linked list with header"
  - Simple code for `insert` / `erase`
  - Use most space (two pointers per node, need header node)
  - Can `push_back`, `pop_back`, `push_front`, `pop_front`
- Also need custom iterator class
  - Iterator just store a pointer to a node

- We cannot directly use a pointer to a node (`node*`) because we need to override some operator (`--`, `++`, and something else)

CP::list

Layout

- Inner class is a class inside another class
  - Inner class can access any members of outer class
  - Outer class cannot access `protected` or `private` of the inner class
- Friend class allows other class to access

```
template <typename T>
class list {
  protected:
    class node {
      friend class list;
      public:
        T data;
        node *prev, *next;
        // some functions
    };
    class list_iterator {
      friend class list;
      protected:
        node* ptr;
      public:
        // some functions && operators
    };
  public:
    typedef list_iterator iterator;
  protected:
    node *mHeader; // pointer to a header node
    size_t mSize;
  public:
    // functions
};
```

Doubly Linked List Node

```
class node {
  friend class list;
  public:
    T data;
    node *prev;
    node *next;

    node() :
      data( T() ), prev( this ), next( this ) { }
    node(const T& data, node* prev, node* next) :
      data( T(data) ), prev( prev ), next( next ) { }
};
```

Constructor

```
  // default constructor
  list() : mHeader( new node() ), mSize( 0 ) { }

  // copy constructor
```

```
  list(list<T>& a) : mHeader( new node() ), mSize( 0 ) {
    for (iterator it = a.begin(); it != a.end(); it++) {
      push_back(*it);
    }
  }

  list<T>& operator=(list<T> other) {
    using std::swap;
    swap(this->mHeader, other.mHeader);
    swap(this->mSize, other.mSize);
    return *this;
  }

  ~this() {
    clear();
    delete mHeader;
  }
```

Small functions

```
  // capacity function
  bool empty() const { return mSize == 0; }
  size_t size() const { return mSize; }

  // access
  T& front() { return mHeader->next->data; }
  T& back() { return mHeader->prev->data; }

  // modifier
  void push_back(const T& element) {
    insert(end(), element);
  }
  void push_front(const T& element) {
    insert(begin(), element);
  }
  void pop_back() {
    erase(iterator(mHeader->prev));
  }
  void pop_front() {
    erase(begin());
  }
```

- Task is delegated to `insert` and `erase`
- Need iterator

Iterator

```
class list_iterator {
  friend class list;
  protected:
    node* ptr;
  public:

  list_iterator() : ptr( NULL ) { }
  list_iterator(node *a) : ptr(a) { }

  list_iterator& operator++() {
    ptr = ptr->next;
```

```
    return (*this);
  }

  list_iterator& operator--() {
    ptr = ptr->prev;
    return (*this);
  }
```

- Has custom constructor that takes node pointer

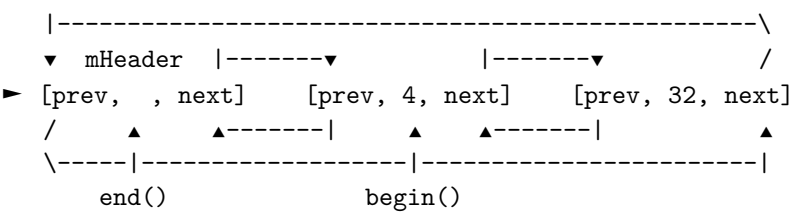
```
  list_iterator& operator++(int) {
    list_iterator tmp(*this);
    operator++();
    return tmp;
  }

  list_iterator& operator--(int) {
    list_iterator tmp(*this);
    operator--();
    return tmp;
  }
```

- `operator++()` is an operator for `++it`
- `operator++(int)` is a syntax for `it++`
- `operator++(int)` delegates to `operator++()`
- Same for `operator--()`

```
  T& operator*() { return ptr->data; }
  T* operator->() { return &(ptr->data); }

  bool operator==(const list_iterator& other) {
    return other.ptr == ptr;
  }

  bool operator!=(const list_iterator& other) {
    return other.ptr != ptr;
  }
};
```

Other small functions

```
iterator begin() {
  return iterator(mHeader->next);
}

iterator end() {
  return iterator(mHeader);
}

void clear() {
  while (mSize > 0) erase(begin());
}
```



Insert & Erase

```
iterator insert(iterator it, const T& element) {
  node *n = new node(element, it.ptr->prev, it.ptr);
  it.ptr->prev->next = n;
  it.ptr->prev = n;
  mSize++;
  return iterator(n);
}

iterator erase(iterator it) {
  iterator tmp(it.ptr->next);
  it.ptr->prev->next = it.ptr->next;
  it.ptr->next->prev = it.ptr->prev;
  delete it.ptr;
  mSize--;
  return tmp;
}
```

- Header make `insert` / `erase` very simple

Lecture 15

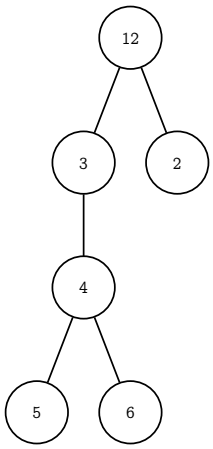
Binary Tree

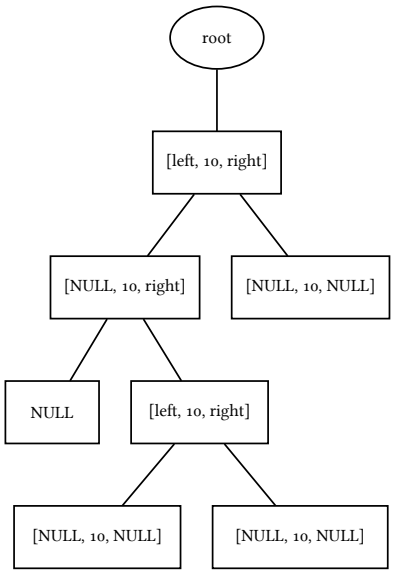
Practicing Pointer & Recursive

Overview

- This is a basic for the next data structure, Binary Search and AVL Tree
- Focus on using Node and Pointer
- Focus on using recursive programming
- Some applications using just Binary Tree
- There is no data in `std` that is Binary Tree

Binary Tree & Node





- A rooted tree where each node have at most two children
- Tree Node is very similar to a linked list node

```
node
  left data right
  NULL  10  NULL
```

```
class node {
  public:
    ValueT data;
    node *left, *right;
    node() :
      data( ValueT() ), left( NULL ), right( NULL ) { }
    node(const ValueT& data, node* left, node* right) :
      data( data ), left( left ), right( right ) { }
};
```

Node with parent link

```
class node {
  public:
    ValueT data;
    node *left, *right, *parent;
    node() :
      data( ValueT() ), left( NULL ), right( NULL ), parent( NULL ) { }
    node(const ValueT& data, node* left, node* right, node* parent) :
      data( data ), left( left ), right( right ), parent( parent ) { }
};
```

- Sometimes, we need a link to parent
- Root is the only node that parent is `NULL`

String Encoding

Huffman Coding: Example Application of Tree

- David Huffman proposed this as his term project in Robert Fano's class (co-worker of Claude Shannon) which beats Shannon-Fano encoding
- Encoding = associate meaning to a representation

- ASCII Code
  - Fix length encoding
  - Each char = 8 bits

Variable Length Encoding

Never gonna give you up
Never gonna let you down
Never gonna run around and desert you

16 different characters
Fix-length needs $4 \times 86 = 344$ bits
Variable Length need 327 bits

n	e	o	u	r	a	v	g	d	y	t	w	s	p	l	i
14	11	9	7	7	6	5	5	5	4	3	2	2	2	2	2
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
11	010	011	0001	0011	0000	1011	1010	1000	0010 1	1001 1	1001 01	0010 001	0010 01	1001 00	0010 000

Encoding “never”
Fix-length 00000001011000010100
Variable-length 11010110110100011

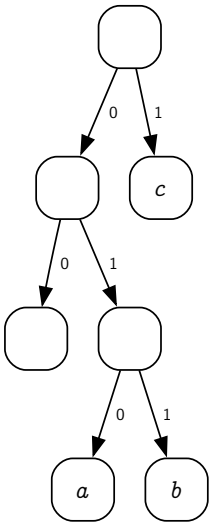
Huffman Coding

Problem Statement

- Input: a string
- Output: encoding of each character in the string such that
  - The total length of encoding is minimum
  - The encoding of each character is *not ambiguous*
    - Any character encoding is not a prefix of any other character

Tree Encoding

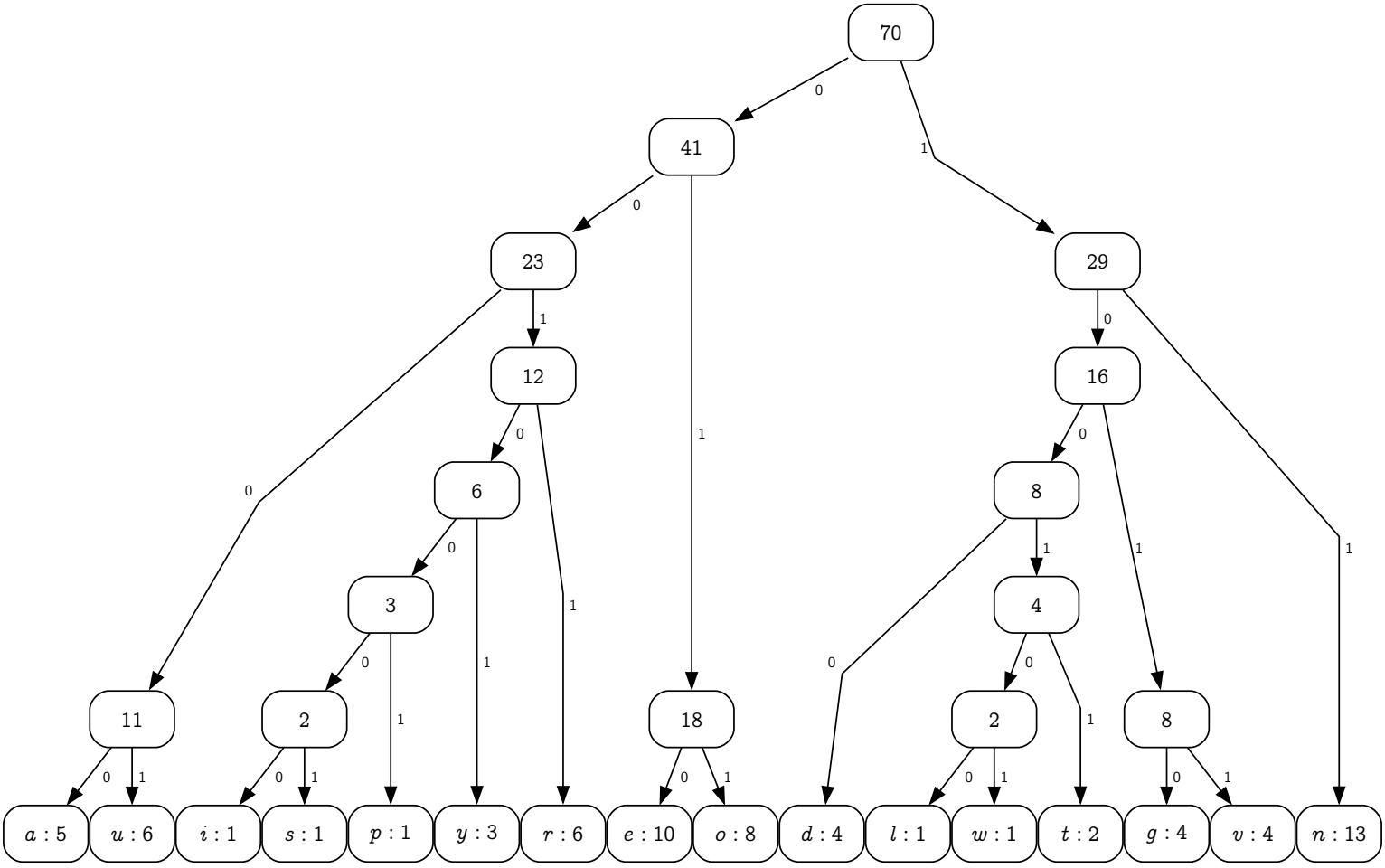
- Using a tree to represent encoding
- Each character is represented at leaf nodes
  - Leaf node is a node without children
- Encode by start at the root and walk toward leaf nodes
  - The path gives the encoding
  - Going to left child equal to 0
  - Going to right child equal to 1
- Guaranteed to be non-ambiguous




```
a = 010
b = 011
c = 1
```

Huffman Tree

- Find 2 min nodes
- Create a new node with those two nodes as children, set freq equal to summation
- Repeat until only one node left



Huffman Tree

Huffman Tree Node

- Instead of data, we have both character and frequency
- Since we have to pick two nodes with minimum freq, we overload `operator<` to do so and use `priority_queue`

Huffman Code: Node

```
class huffman_tree {
  protected:
    class huffman_node {
      public:
        char letter;
        int freq;
        huffman_node *left, *right;
        huffman_node() : letter('*'), freq(0), left(NULL), right(NULL) {}
        huffman_node(char letter, int freq, huffman_node *left, huffman_node *right) :
          letter(letter), freq(freq), left(left), right(right) {}

        bool is_leaf() { return left == NULL && right == NULL; }
    };
```

```
    class node_comparator {
      public:
        bool operator()(const huffman_node *a, huffman_node *b) {
          return a->freq > b->freq;
        }
    };
};
```

Huffman Code: Build Tree

```
class huffman_tree {
  protected:
    huffman_node *root;
    void build_tree(vector<huffman_node*> data) {
      priority_queue<huffman_node*, vector<huffman_node*>, node_comparator> pq;
      for (auto &x : data) pq.push(x);
      while (pq.size > 1) {
        huffman_node *right = pq.top(); pq.pop();
        huffman_node *left = pq.top(); pq.pop();
        pq.push(new huffman_node('*', left->freq+right->freq, left, right));
      }
      root = pq.top();
    }
  public:
    huffman_tree(string s) {
      map<char, int> count;
      for (auto &c : s) {
        count[c]++;  // word count
      vector<huffman_node*> nodes;
      for (auto &x : count)
        nodes.push_back(new huffman_node(x.first, x.second, NULL, NULL));
      build_tree(nodes);
      }
    }
};
```

Recursive Programming

Calling itself

Recursive

- A function that call itself
- Must have some input, usually via function argument
- The function must check a condition for execution
  - Result in either terminating case where the function won't call itself
  - or recursion case where the function will call itself with different parameters

```
// calculate sum 0..n
int recur1(int n) {
  if (n <= 0) {
    // terminating case
    return 0;
  } else {
    // recursion case
    return recur1(n-1) + n;
  }
}
```

Why recursion?

- Much simpler code
  - When the task is right
  - Recursion is natural for several mathematical model that is recursi
- Comparing to a normal loop, recursion has the same growth rate but recursion might take more time because function call is costlier than a loop

More Example

```
void print_range1(int step, int goal) {
  if (step < goal) {
    std::cout << step << "";
    print_range1(step+1, goal);
  }
}
```

- Terminating Case do nothing
- Which is the output of `print_range1(0,5)` and `print_range2(0,5)`

```
void print_range2(int step, int goal) {
  if (step < goal) {
    print_range2(step+1, goal);
    std::cout << step << "";
  }
}
```

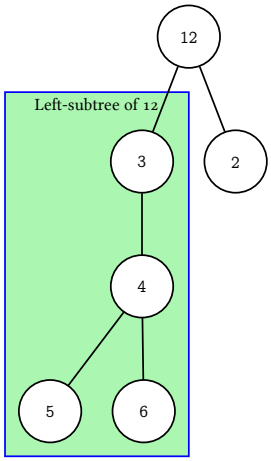
1. `0 1 2 3 4 5`
2. `0 1 2 3 4`
3. `5 4 3 2 1 0`
4. `4 3 2 1 0`

Binary Tree Recursive Definition

- A Binary Tree is
  - A tree with no nodes (root is `NULL`)
  - A tree with a root
    - Both children of the root must be a binary tree
    - Each child is called left-subtree and right-subtree
- Since binary tree can be defined recursively, operation on a binary tree can be naturallly written as a recursion

Subtree

- For any node
  - its left (or right) child and all of the child's descendants is called left-subtree (or right-subtree)



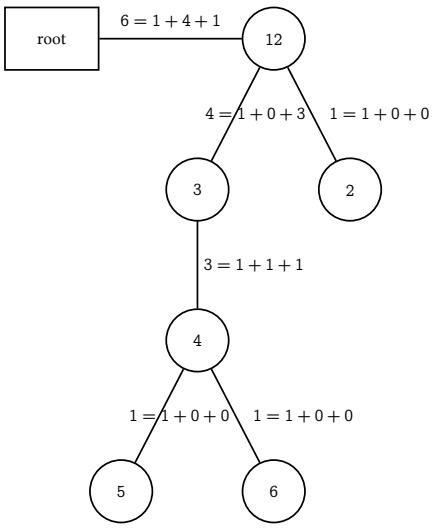
Recursion on Binary Tree

Tree Size by Recursion

- An empty tree has 0 nodes

- A tree with a root has 1 node (the root)
  - Plus the size of its two subtrees
- Easily written as recursive

```
class node {
  public:
    int data;
    node *left, *right;
};

int get_size(node* n) {
  if (n == NULL) return 0;
  return 1 + get_size(n->left) + get_size(n->right);
}
```



Tree Height

- Height of a tree is the number of link we have to go to reach it deepest children
- Empty tree has height `-1`
- Height of a tree is `1 + max` of height of its children

```
class node {
  public:
    int data;
    node *left, *right;
};

int get_height(node* n) {
  if (n == NULL) return 0;
  return 1 + std::max(get_height(n->left),
                      get_height(n->right));
}
```

Tree Copy

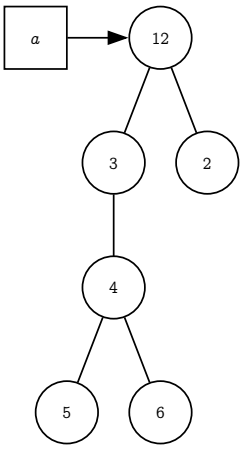
```
class node {
  public:
    int data;
    node *left, *right;
```

```
    node() : data(0), left(NULL), right(NULL);
    node(int data, node *left, node *right)
      : data(data), left(left), right(right);
};

node* copy(node *n) {
  if (n == NULL) return NULL;
  node *lc = copy(n->left);
  node *rc = copy(n->right);
  node *result = new node(n->data, lc, rc);
}
```

Walk over a tree

- Visiting all nodes (and maybe do something)

```
// Preorder traversal
void preorder(node *n) {
  if (n == NULL) return;
  std::cout << n->data << " ";
  preorder(n->left);
  preorder(n->right);
}

// Inorder traversal
void inorder(node *n) {
  if (n == NULL) return;
  inorder(n->left);
  std::cout << n->data << " ";
  inorder(n->right);
}

// Postorder traversal
void postorder(node *n) {
  if (n == NULL) return;
  postorder(n->left);
  postorder(n->right);
  std::cout << n->data << " ";
}
```



What is the result of

- `preorder(a)`
  - `12 3 4 5 6 2`
- `inorder(a)`
  - `5 4 6 3 12 2`
- `postorder(a)`
  - `5 6 4 3 2 12`

Huffman Tree: Encoding

```
class huffman_tree {
  protected:
    class huffman_node { };
    class node_comparator { };
    huffman_node *root;
  public:
    void print(huffman_node *n, string s) {
      if (n->is_leaf()) {
        cout << n->letter << ": " << s << endl;
      } else {
        print(n->left,s+"0");
        print(n->right,s+"1");
      }
    }

    void print() {
```

```
      print(root,"");
    }
};
```

- Recursive printing
- Use `s` to store path

```
class huffman_tree {
  protected:
    class huffman_node { };
    class node_comparator { };
    huffman_node *root;

    void delete_node(huffman_node *n) {
      if (n == NULL) return;
      delete_node(n->left);
      delete_node(n->right);
      delete n;
    }

  public:

    ~huffman_tree() {
      delete_node(root);
    }
};
```

- Recursive delete node
- Use postorder traversal
- Can we use inorder or preorder?

Lecture 16

Binary Search Tree

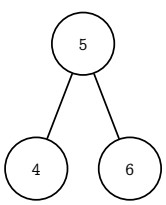
Binary Tree with value condition

Overview

- We add additional value constraint to a Binary Tree
- The constraint make finding data in the tree much faster
  - $\mathcal{O}(h)$ where h is the height of the tree
  - The tree is expected to have h be in $\mathcal{O}(\lg n)$, but this is not always true
  - The next tree (AVL tree) will add more constraint so that we can guarantee that $h = \mathcal{O}(\log n)$
- Using the same approach as a binary heap, maintain the constraint during modification

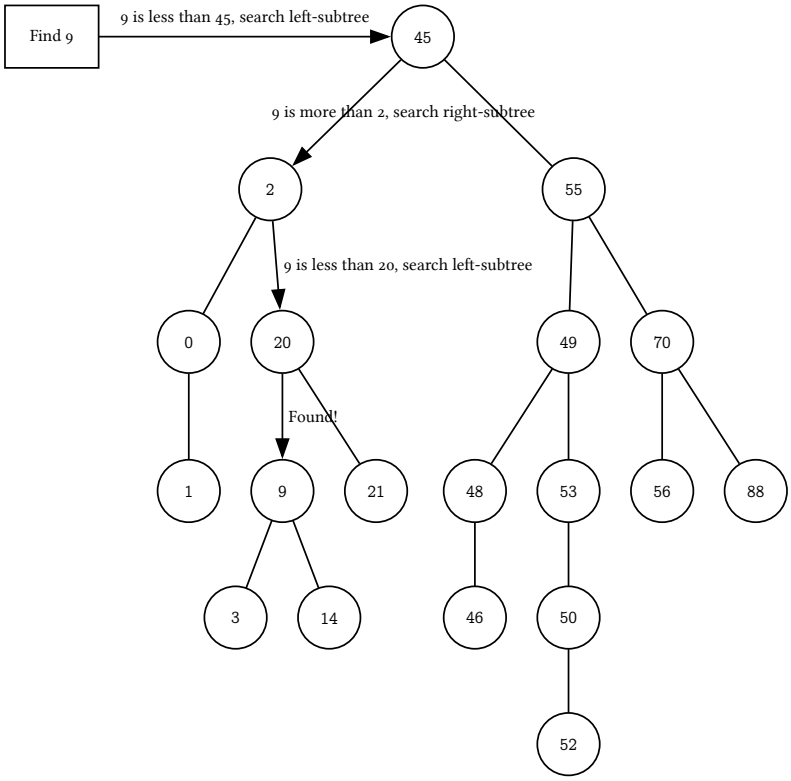
Binary Search Tree

- Structure rule: must be a Binary Tree
- Value rule: for any node `x`
  - data in left-subtree must be less than the data in `x`
  - data in right-subtree must be more than the data in `x`
- Recursive Definition
  - An empty tree is a Binary Search Tree (BST)
  - A node `X` is a BST when
    - Its subtrees (if any) must be BST and
    - If left-subtree exists, `X->data` must be more than `X->left->data`
    - If right-subtree exists, `X->data` must be less than `X->right->data`



Finding Value in BST

- Value rules make finding fast
- To find `e`, start from root
  - If the current node is not `e`,
    - search in left-subtree if `e` is less than the current node
    - search in right-subtree if `e` is more than the current node
  - Keep going until we find `e` or reach `NULL`
- Other operation is also depends on `find`



Insert && Erase with 0, 1 child

Find Node

```
node* find_node(const ValueT& k, node* r, node* &parent) {
  node *ptr = r;
  while (ptr != NULL) {
    int cmp == compare(k, ptr->data);  // return -1 if a < b, 1 if a > b, 0 if equal
    if (cmp == 0) return ptr;
    parent = ptr;
    ptr = cmp < 0 ? ptr->left : ptr->right;
  }
  return NULL;
}
```

Insert

- Assumption: Data in BST is unique
- `insert(e)` by find `e`
  - If `e` is found, don't add any node
  - If `e` is not in BST, find must reach `NULL` somewhere, that `NULL` is where to put `e`
- Both structure and value constraints are satisfied

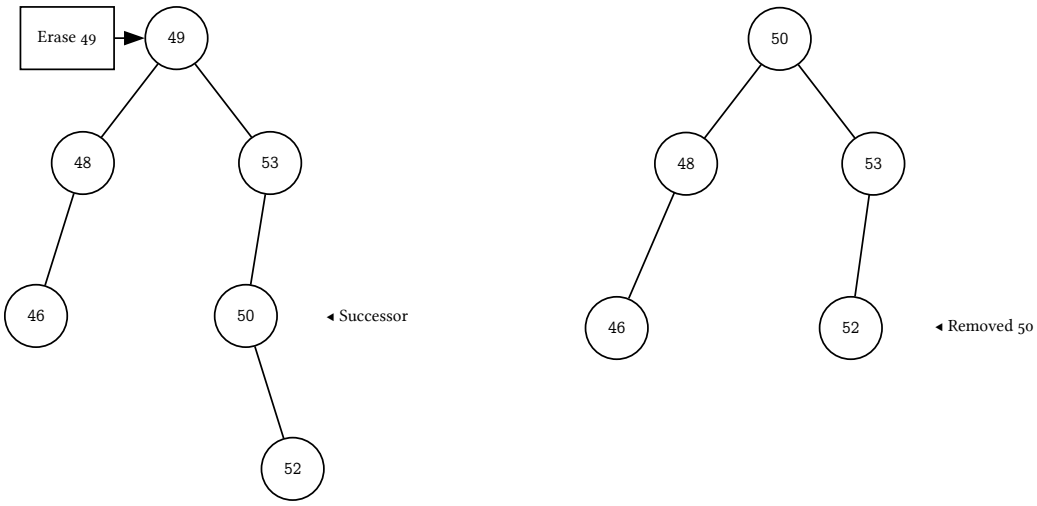
Erase

- `erase(e)` first have to find `e` as well
- If not found, do nothing
- If found at node `X`, there are 3 cases depend on the number of children of `e`
  - If has no child, just simply `delete X`
  - If has one child, have parent of `X` points (using the same link) to the child of `X` instead
  - If has two children, pick either successor or predecessor of `e`
    - Assume we choose successor `p` (must be in right-subtree), replace `X` with `p` and `erase(p)` from right subtree

Erase with 2 children

Erase node with 2 children

- Replace by successor (or predecessor) preserves value rules
  - Successor is the minimum in the right-subtree
  - Predecessor is the maximum in the left-subtree
- Both exists (because the node has both subtrees)



Find min max

Finding Successor and Predecessor

- If a tree has left-subtree, `min` is the min of left-subtree
  - If not, `min` is the root

```
node* find_min_node(node* r) {
  // r must not be NULL
  node *min = r;
  while (min->left != NULL) {
    min = min->left;
  }
  return min;
}
```

- If a tree has right-subtree, `max` is the max of right-subtree
  - If not, `max` is the root

```
node* find_max_node(node* r) {
  // r must not be NULL
  node *max = r;
  while (max->right != NULL) {
    max = max->right;
  }
  return max;
}
```

Finding Successor and Predecessor (recursive)

```
node* find_min_node(node* r) {
  // r must not be NULL
  if (r->left == NULL) return r;
  return find_min_node(r->left);
}
```

```
node* find_max_node(node* r) {
  // r must not be NULL
  if (r->right == NULL) return r;
  return find_max_node(r->right);
}
```

Complexity Analysis

- `insert`, `erase` depends on `find`, `find_min` (or `find_max`)
- All finds start from root and in the worst case reach the leaf
  - Hence, $\mathcal{O}(h)$
- Height of the tree can be in the range from $\lg n$ to n
- For $1,000,000$ nodes, it's in the range of $[20, 999999]$
  - $\mathcal{O}(h)$ is, right now, $\mathcal{O}(n)$
  - Will be fixed by AVL tree

CP::map_bst (insert)

Using Binary Search Tree to create associated data structure

Layout

- Need node class
- Also need iterator class
- Template has two types
  - Key Type and Mapped Type
  - ValueType is `pair<KeyType, MappedType>`
- Also need custom comparator

```
template <typename KeyT,
          typename MappedT,
          typename CompareT = std::less<KeyT> >
class map_bst {
  protected:
    typedef std::pair<KeyT,MappedT> ValueT;
    class node {
      friend class map_bst;
      protected:
        ValueT data;
        node  *left;
        node  *right;
        node  *parent;
    };
    class tree_iterator {
      protected:
        node* ptr;
      public:
    };
    node    *mRoot;
    CompareT mLess;
    size_t   mSize;
  public:
    typedef tree_iterator iterator;
};
```

Node class

- Data stores both the key type and mapped type (as a pair)

- Map finds by key

```
class node {
  friend class map_bst;
  protected:
    ValueT data;
    node  *left;
    node  *right;
    node  *parent;

  node() :
    data( ValueT() ), left( NULL ), right( NULL ), parent( NULL ) { }

  node(const ValueT& data, node* left, node* right, node* parent) :
    data( data ), left( left ), right( right ), parent( parent ) { }
};
```

Ctors, Dtor

```
map_bst(const map_bst<KeyT, MappedT, CompareT> & other) :
  mLess(other.mLess), mSize(other.mSize)
{ mRoot = copy(other.mRoot, NULL); }  // recursive copy

map_bst(const CompareT& c = CompareT() ) :
  mRoot(NULL), mLess(c), mSize(0)
{ }

map_bst<KeyT, MappedT, CompareT>& operator=(map_bst<KeyT, MappedT, CompareT> other) {
  using std::swap;
  swap(this->mRoot, other.mRoot);
  swap(this->mLess, other.mLess);
  swap(this->mSize, other.mSize);
  return *this;
}

~map_bst() {
  clear();  // recursive delete
}
```

Actual Find

```
iterator find(const KeyT &key) {
  node *parent;
  node *ptr = find_node(key, mRoot, mParent);
  return ptr == NULL ? end() : iterator(ptr);
}
```

- Find by Key

```
int compare(const KeyT& k1, const KeyT& k2) {
  if (mLess(k1, k2)) return -1;
  if (mLess(k2, k1)) return 1;
  return 0;
}

node* find_node(const KeyT& k, node* r, node* &parent) {  // modify parent pointer
  node *ptr = r;
  while (ptr != NULL) {
    int cmp = compare(k, ptr->data.first);
```

```
    if (cmp == 0) return ptr;
    parent = ptr;
    ptr = cmp < 0 ? ptr->left : ptr->right;
  }
  return NULL;
}
```

Insert

- `insert` returns pair of iterator and insert result

```
std::pair<iterator, bool> insert(const ValueT& val) {
  node *parent = NULL;
  node ptr* = find_node(val.first, mRoot, parent);
  bool not_found = (ptr==NULL);
  if (not_found) {
    ptr = new node(val, NULL, NULL, parent);
    child_link(parent, val.first) = ptr;
    mSize++;
  }
  return std::make_pair(iterator(ptr), not_found);
}
```

`child_link` returns a reference (the variable) to the pointer of the appropriate child of the parent with respect to `k`

```
node* &child_link(node* parent, const KeyT& k) {
  if (parent == NULL) return mRoot;
  return mLess(k, parent->data.first) ?
          parent->left : parent->right;
}
```

CP::map_bst (erase and iterator)

Erase

- Handle multiple cases

```
size_t erase(const KeyT &key) {
  if (mRoot == NULL) return 0;  // empty tree
  node *parent = NULL;
  node *ptr = find_node(key, mRoot, parent);
  if (ptr == NULL) return 0;  // not found
  if (ptr->left != NULL && ptr-> != NULL) {
    // have two children => erase min
    node *min = find_min_node(ptr->right);  // successor
    // remove min like case 0 or 1 child
    node * &link = child_link(min->parent, min->data.first);
    link = (min->left == NULL) ? min->right : min->left;
    if (link != NULL) link->parent = min->parent;
    // swap min with ptr
    std::swap(ptr->data.first, min->data.first);
    std::swap(ptr->data.second, min->data.second);
    ptr = min;  // we are going to delete this node instead
  } else {
    // case 0 or 1 child => erase ptr
    node * &link = child_link(ptr->parent, key); // find the pointer that holds ptr
    link = (ptr->left == NULL) ? ptr->right : ptr->left;  // replace it with ptr child
    if (link != NULL) link->parent = ptr->parent;  // if child exists, update parent
  }
```

```
  delete ptr;
  mSize--;
  return 1;
}
```

Operator[]

```
MappedT& operator[](const KeyT& key) {
  node *parent = NULL;
  node *ptr = find_node(key, mRoot, parent);
  if (ptr == NULL) {
    ptr = new node(std::make_pair(key, MappedT()), NULL, NULL, parent);
    child_link(parent, key) = ptr;
    mSize++;
  }
  return ptr->data.second;
}
```

- Find node
- If not exists, create one with default `MappedTypeReturn MappedType` of the node

Iterator

- Just like linked list, we need a class for iterator
  - Because we need custom `operator++`, `--` (and some more)
- Iterator class just store a pointer to a node

```
class tree_iterator {
  protected:
    node* ptr;

  public:
    tree_iterator() : ptr( NULL ) { }
    tree_iterator(node *a) : ptr(a) { }
    // more functions below
};
```

Other Functions

```
tree_iterator operator++(int) {  // ++it
  tree_iterator tmp(*this);
  operator++();
  return tmp;
}

tree_iterator operator--(int) {  // --it
  tree_iterator tmp(*this);
  operator--();
  return tmp;
}

ValueT& operator*() { return ptr->data; }
ValueT* operator->() { return &(ptr->data); }
bool    operator==(const tree_iterator& other)
  { return other.ptr == ptr; }
bool    operator!=(const tree_iterator& other)
  { return other.ptr != other }
```

Operator++

- Find successor of x, easy if x have right-subtree

  - Just find min of right-subtree
- If not, we have to go up (go toward root) until we find one that is more than `x`
  - This is always the closest ancestor of `x` that has `x` in its left-subtree

```
tree_iterator& operator++() {
  if (ptr->right == NULL) {
    node *parent = ptr->parent;
    while (parent != NULL &&
           parent->right == ptr) {
      ptr = parent;
      parent = ptr->parent;
    }
    ptr = parent;
  } else {
    ptr = ptr->right;
    while (ptr->left != NULL)
      ptr = ptr->left;
  }
  return (*this);
}
```

Summary

- Binary Search Tree relies on Value Constraint to make `find` fast
  - Possible to be slow (will be fixed later)
- Erase requires `find_min`, `find_max`
- `CP::map_bst` uses pair to store `KeyT` and `MappedT`
  - `find` uses `key`
- Iterator is just a pointer
  - Have a problem of `operator--` at `end()` (will be fixed later)

Lecture 17

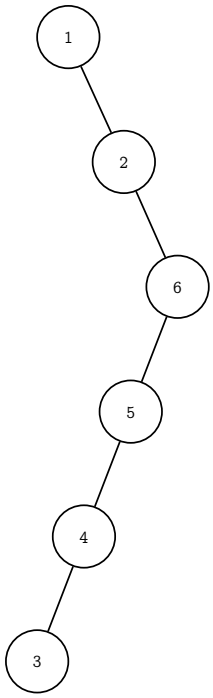
AVL Tree

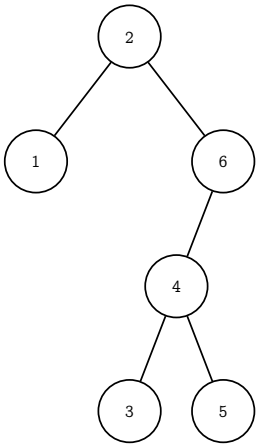
หัวข้อ

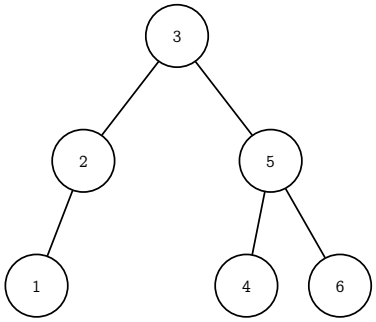
- นิยามต้นไม้เอวีแอล
- การวิเคราะห์ความสูงของต้นไม้เอวีแอล
- การปรับต้นไม้เอวีแอลให้สูงสมดุล
- กระบวนการหมุนปม

ต้นไม้ค้นหาแบบทวิภาค

- เวลาการทำงานเป็น $\mathcal{O}(h)$
- $\lfloor \log_2 n \rfloor \leq h \leq n - 1$
- โชคดีทำงานเร็ว $\mathcal{O}(\log n)$, โชคร้ายทำงานเป็น $\mathcal{O}(n)$

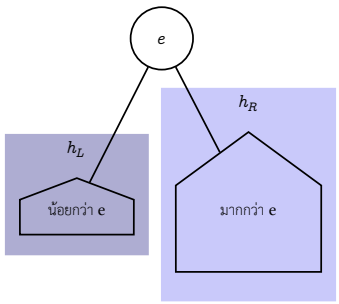






ต้นไม้เอวีแอล

- AVL = Binary Search Tree + กฎความสูงสมดุล

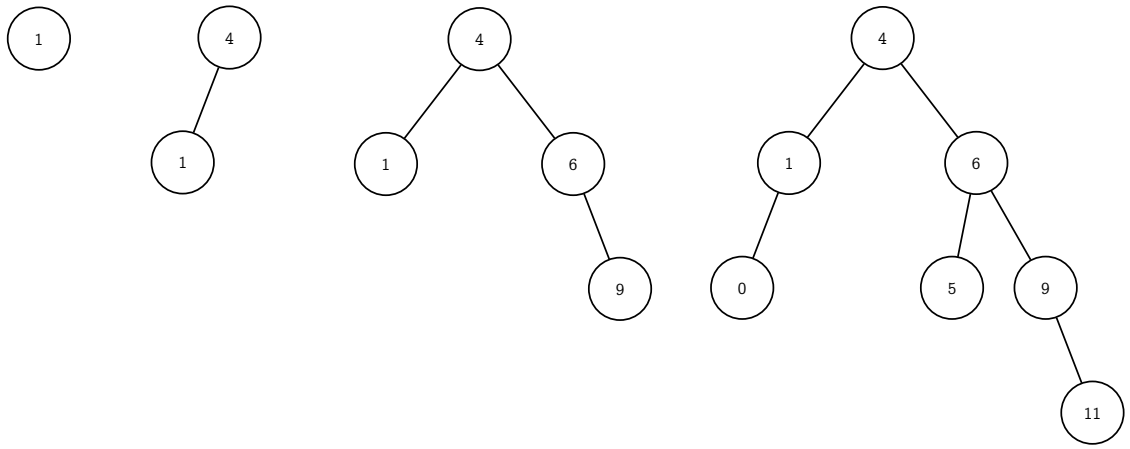


$$|h_L - h_R| \leq 1$$

ต้นไม้ย่อยทุกต้นต้องเป็นไปตามกฎ

AVL: Adelson-Velskii and Landis

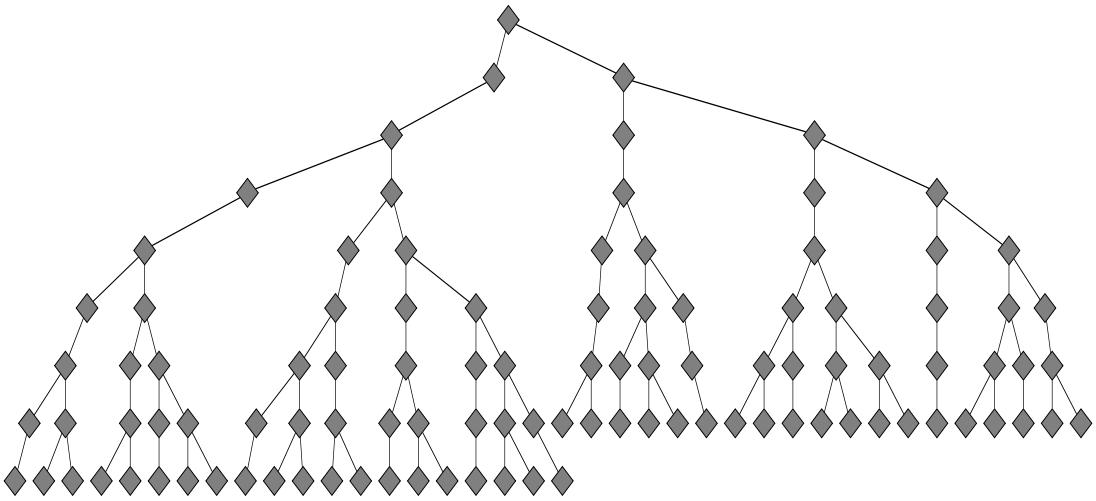
ตัวอย่างต้นไม้เอวีแอล



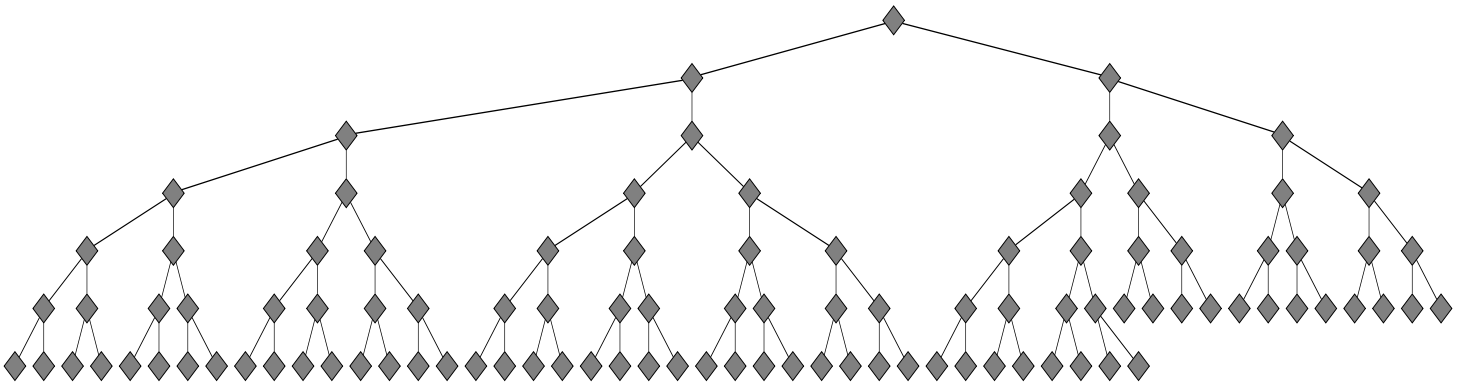
ต้นไม้ว่าง (null) สูง −1

AVL Balance Rule

ต้นไม้ค้นหาแบบทวิภาคเอวีแอล



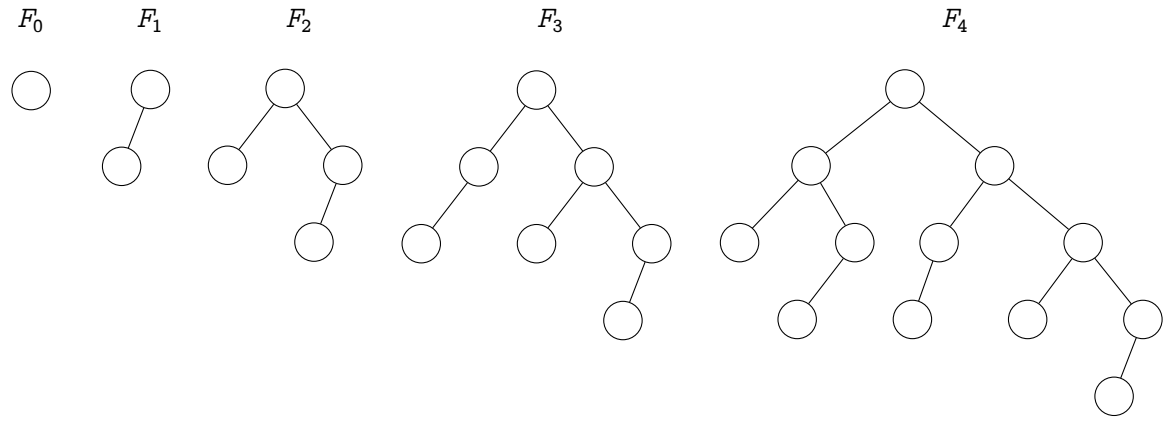
ต้นไม้ค้นหาแบบทวิภาคที่สร้างจากข้อมูลสุ่ม 100 ตัว



ต้นไม้เอวีแอลที่สร้างจากข้อมูลสุ่ม 100 ตัว

ต้นไม้เอวีแอลสูงเท่าใด ?

- ให้ F_n คือต้นไม้เอวีแอลซึ่งสูง h ที่มีจำนวนปมน้อยสุด



$$|F_h| = 1 + |F_{h-1}| + |F_{h-2}|$$

Fibonacci Tree

ความสูงของต้นไม้ฟิโบนักชี

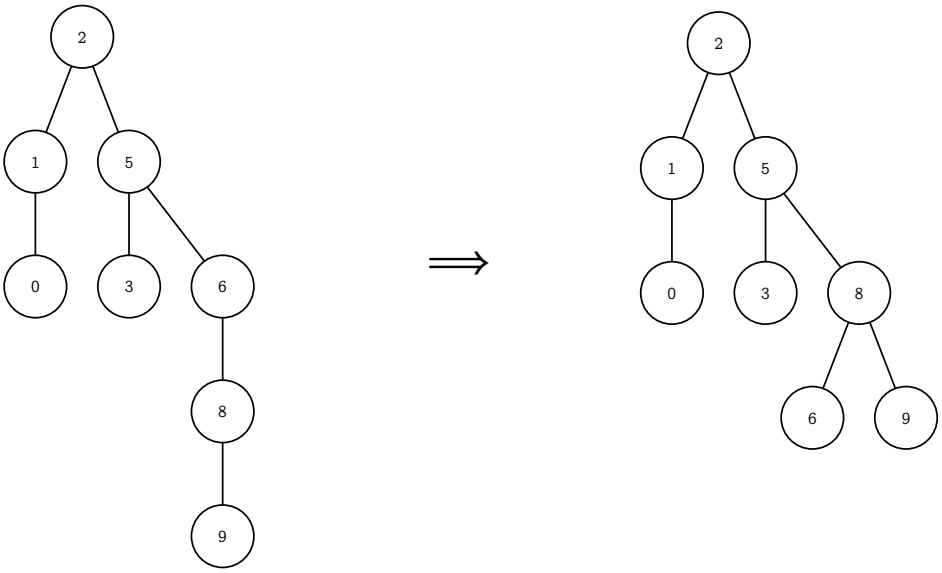
$$\begin{aligned}
|F_h| &= 1 + |F_{h-1}| + |F_{h-2}| \\
n_h &= 1 + n_{h-1} + n_{h-2} \quad h \geq 2, \quad n_0 = 1, \quad n_1 = 2 \\
n_h &= \alpha_1 \phi^h + \alpha_2 \hat{\phi}^h - 1, \quad \phi = 1.618..., \quad \hat{\phi} = -0.618 \\
n_h &\approx \alpha_1 \phi^h \\
h &\approx \frac{1}{\log_2 \phi}(\log_2 n_h) \\
h &\approx 1.44(\log_2 n_h)
\end{aligned}$$

สรุป: ต้นไม้เอวีแอลที่มี n ปม สูงไม่เกิน $1.44 \log_2 n$

AVL Rotation

ทำอย่างไรให้เป็นไปตามกฎของ AVL

- การเพิ่ม/ลบข้อมูลทำเหมือน BSTree
- แต่หลังการเพิ่ม/ลบ อาจทำให้ผิดกฎสูงสมดุล
- ถ้าผิดกฎต้องปรับต้นไม้



map_avl

```
template <typename KeyT,
          typename MappedT,
          typename CompareT = std::less<KeyT> >
class map_avl {
  protected:
    class node {
      friend class map_bst;
      ...
    };

    class tree_iterator {  // เหมือน map_bst
      ...
    };
  public:
    ...
};
```

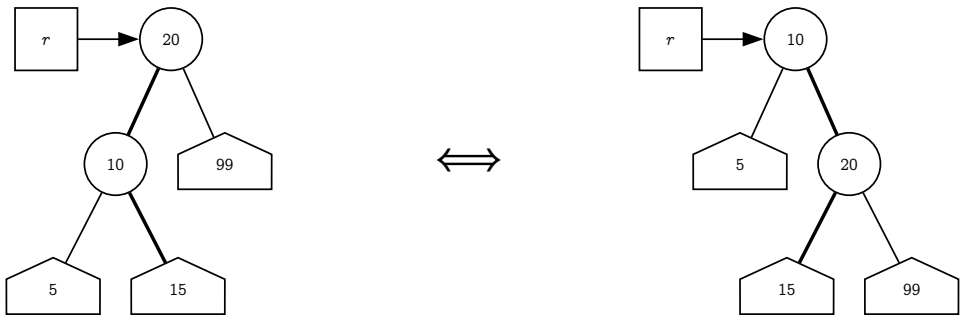
node

```
class node {
  friend class map_avl;
  protected:
    ValueT data;
    node *left;
    node *right;
    node *parent;
    int   height;  // แต่ละปมมีความสูงกำกับ

    node() :
      data( ValueT() ), left( NULL ), right( NULL ),
      parent( NULL ), height(0) { }

    node(const ValueT& data, node* left,
         node* right, node* parent) : data(data),
         left(left), right(right), parent(parent) {
      set_height();
```

```
    }

    int get_height(node *n) {  // ไม่ oo ?
      return (n == NULL ? -1 : n->height);
    }
    void set_height() {
      int hL = get_height(this->left);
      int hR = get_height(this->right);
      height = 1 + (hL > hR ? hL : hR);
    }
    int balance_value() {
      return get_height(this->right) -
             get_height(this->left);
    }

    void set_left(node *n) {
      this->left = n;
      if (n != NULL) this->left->parent = this;
    }
    void set_right(node *n) {
      this->right = n;
      if (n != NULL) this->right->parent = this;
    }
};
```

การหมุนปม

- การปรับต้นไม้อาศัยการหมุน (rotation)



`rotate_left_child(r)`
$h_r = \max(2+5, 2+15, 1+99)$

`rotate_right_child(r)`
$h_r = \max(1+5, 2+15, 2+99)$

CP::map_avl

rotate_left_child(r)

```
node *rotate_left_child(node *r) {
  node *new_root = r->left;
  r->set_left(new_root->right);
  new_root->set_right(r);

  new_root->right->set_height();
  new_root->set_height();
```

```
  return new_root;
}
```

rotate_right_child(r)

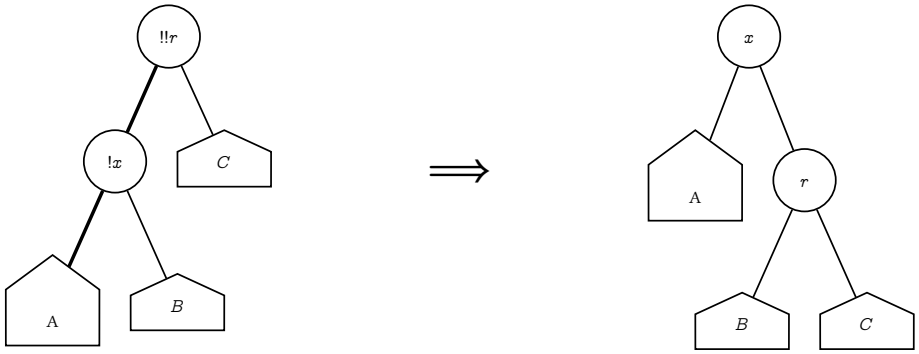
```
node *rotate_right_child(node *r) {
  node *new_root = r->right;
  r->set_right(new_root->left);
  new_root->set_left(r);

  new_root->left->set_height();
  new_root->set_height();
  return new_root;
}
```

insert และ erase ใช้ rebalance

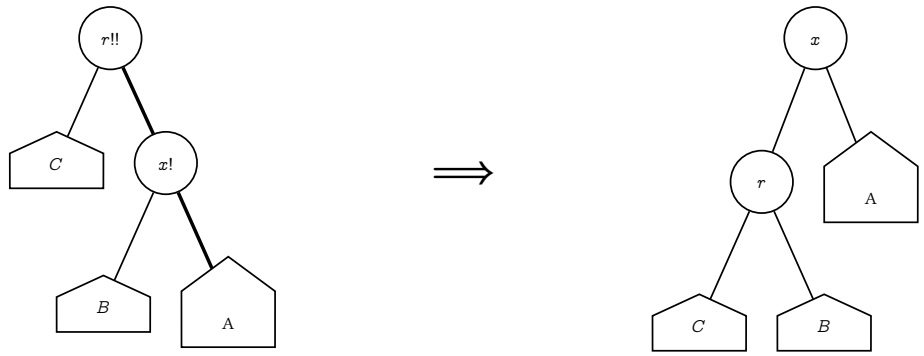
```
node* insert(const ValueT& val, node *r, node * &ptr) {
  ... // same as insert in map_bst

  r = rebalance(r);  // เพิ่มตามปกติ แล้วค่อยปรับ
  return r;
}

node *erase(const KeyT &key, node *r) {
  ... // same as erase in map_bst

  r = rebalance(r);  // ลบตามปกติ แล้วค่อยปรับ
  return r;
}
```

rebalance

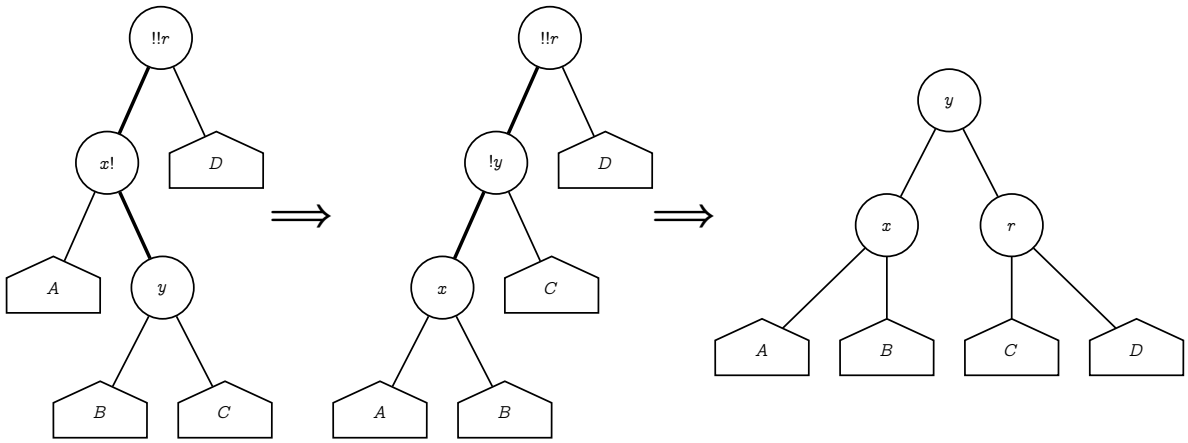
1. `r` เอียงซ้ายมากไป และ `x` เอียงซ้าย ⇒ `rotate_left_child(r)`



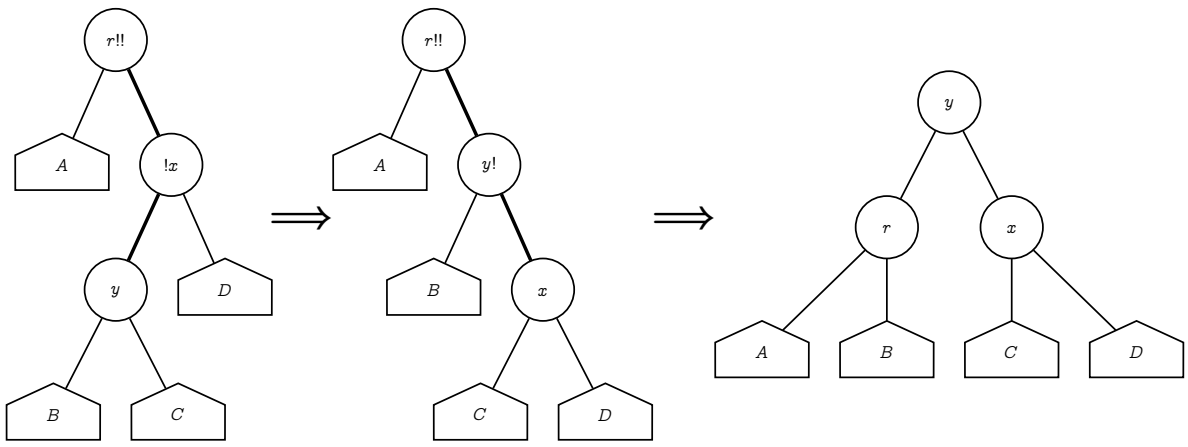
2. `r` เอียงขวามากไป และ `x` เอียงขวา ⇒ `rotate_right_child(r)`



3. r เอียงซ้ายมากไป และ x เอียงขวา ⇒ `rotate_right_child(r->left)` `rotate_left_child(r)`



4. r เอียงขวามากไป และ x เอียงซ้าย ⇒ `rotate_left_child(r->right)` `rotate_right_child(r)`

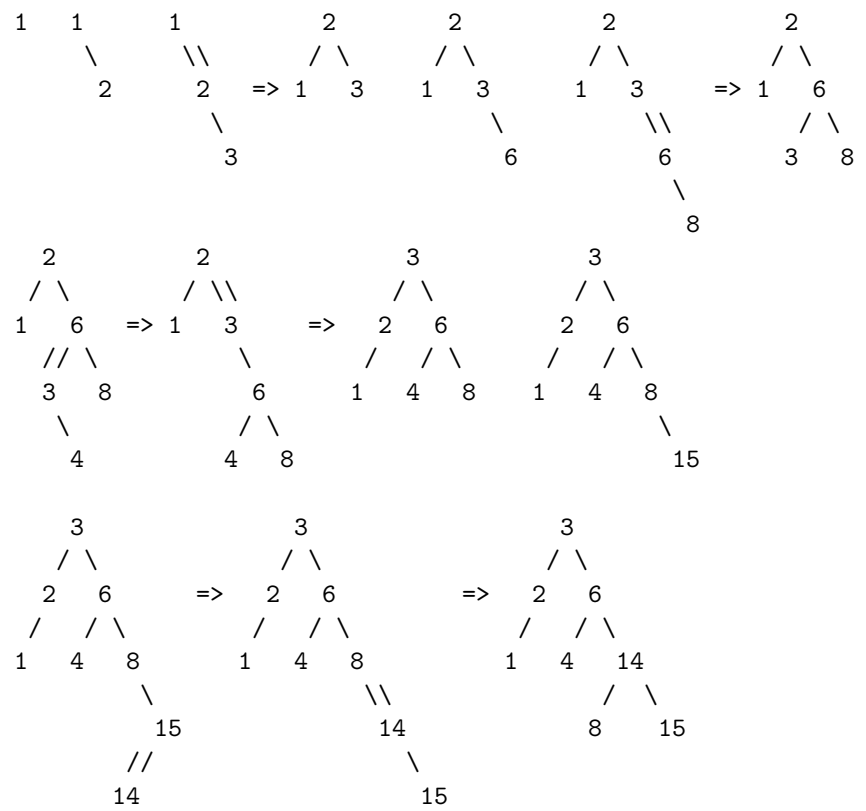



```
node *rebalance(node *r) {
  if (r == NULL) return r;
  int balance = r->balance_value();
  if (balance == -2) {
    if (r->left->balance_value() == 1)  // case 3
```

```
      r->set_left(rotate_right_child(r->left));
    r = rotate_left_child(r);
  } else if (balance == 2) {
    if (r->right->balance_value() == -1)  // case 4
      r->set_right(rotate_left_child(r->right));
    r = rotate_right_child(r);
  }
  r->set_height();
  return r;
}
```

ตัวอย่าง

```
    1, 2, 3, 6, 8, 4, 15, 14
```



สรุป

- ต้นไม้เอวีแอลคือต้นไม้ค้นหาที่ถูกควบคุมความสูง
- ผลต่างความสูงของลูกสองข้างห้ามเกินหนึ่ง
- พิสูจน์ได้ว่า $\lfloor \log_2 n \rfloor \leq h < 1.44 \log_2 n$
- แต่ละปมเก็บความสูงไว้ตรวจสอบ
- ถ้าหลังเพิ่ม/ลดข้อมูลแล้วผิดกฎ ให้ปรับต้นไม้
- การปรับต้นไม้อาศัยการหมุนปม
- เวลาการทำงานของการเพิ่ม ลง และค้นหาเป็น $\mathcal{O}(\log n)$

ความสูงของต้นไม้ AVL

```
class node {
  friend class map_avl;
  protected:
    ValueT data;   // int, int 8
    node  *left;   // 4
    node *right;   // 4
```

```
class node {
  friend class map_avl;
  protected:
    ValueT data;
    node  *left;
    node *right;
```

```
    node *parent;  // 4
    int   height;  // 4
}
```

```
    node *parent;
    unsigned char  height;  // 1
}
```

นอกเรื่อง

- `std::map` ใช้ Red Black Tree
- self-balanced tree มี AVL, Red Black, Splay

Lecture 18

Hash Table

Hash Function

CP::unordered_map

Separate Chaining Iterator

Open Addressing

Linear Probing Iterator

Liner Probing Hash Table

Quadratic Probing

Double Hashing